26S_CST8916_450 Remote Data and RT Applications

Assignment 1

Introduction to Web Services-Based API Architectures

Student Name: Jey Singh

Student ID: 41287569

Instructor: Sourav

Date: May 17, 2026

Introduction to Web Services-Based API Architectures

Question 1: You have been assigned the task of developing a REST API for a popular e-commerce platform. Can you outline a scenario where adhering to the fundamental principles of REST would enhance the user experience and scalability of the system?

Suppose you have to develop a REST API for a big online store such as Amazon or eBay with tho usands of products and users at a time. Amazon or eBay is selling thousands of products everyd ay, serving millions of customers, and taking many orders without crashing or slowing down. REST, or Representational State Transfer, gives us rules for implementing this API. The first, and probably the most significant, is that the API must be stateless.

What this basically means is that each and every request the client makes to the server must contain all the information the server needs to successfully process the request, e.g. who the client is and what they want. The server retains no information about prior requests. This may sound strange at first, but it will be advantageous in that, during load increases like during a Black Friday sale, new servers can be brought online when required, each capable of responding to any incoming request without any need to access previous session information.

Next is the uniform interface principle, which, at the highest level, expresses the aspiration to keep things simple, predictable, and uniform. For instance, if a product, order, or user account exists within amazon system, then each one of those should have its resource and hence its own URL, such as www.amazon.ca/products/iPhone17ProMax or /orders/1100376456.

Cacheability is another principle. When a customer views, say, a product page, it is highly improbable that the product details will have changed since the last time the customer viewed that page. Ideally, such data should be cached either within the browser or the application to which such requests are made, avoiding repeated requests to the server. On subsequent visits, the cached copy will be displayed instantaneously. This not only reduces the load on the server but also allows the customer to browse the site faster.

Then there is the layered system constraint, which allows the security team to position security layers anywhere between the client and the server to secure the application without the user ever knowing. Introduced all together, the discussed REST architectural constraints will add long term stability, scalability, and simplicity to the architecture design of your e-commerce platform.

Question 2:You are working on a project that requires implementing various HTTP methods in a RESTful API. Could you describe a scenario where the proper utilization of each HTTP method significantly improves the efficiency and functionality of the API?

The HTTP methods are actions that a RESTful API can perform. Ensuring that a RESTful API uses the appropriate methods leads to less disorder and confusion. Emblematic methods used are mainly GET, POST, PUT, and DELETE, while each carries out a specified function. To give a practical understanding, these will be illustrated using an example of a food delivery app.

A GET request will be made whenever the user is browsing, for example as soon as the user opens the RESTful app, the RESTful app performs a GET request to retrieve the restaurant's menu. A GET request means the app asks the RESTful server to send some data but not change anything on the RESTful server side. Since GET requests do not change states, they are safe to repeat as many times as the user wishes. Because these requests can be cached, going back to a restaurant previously viewed might be loaded from the cache rather than making a slight RESTful call to the server. This makes a faster restaurant view and reduces the server load.

On the other hand, POST requests are functionalities performed for an order request. You choose your pizza, input your address where it should be delivered, and so go on with your order payment details, confirming this should be done with the utmost certainty. This information application sends to the RESTful server for such data to be mediated using a POST request. In response, the RESTful server should accept it and create a brand new order that was not previously on record, then return a confirmation response. POST is especially used to create new records in the system; every time it is used, new data is added inside the entity.

PUT is used to replace the representation of an existing resource in its entirety. Let's suppose there was an initial mistake: the pizza size you ordered was wrong and you'd like to change the toppings and delivery address as well. Then, you'd issue one PUT request with all the new details, and the RESTful server would replace the old order request with the entity you just sent over. Again, none of the old ones will be stored unless they are present once again in that request.

And, finally, the DELETE method deletes that instance from the server-side application. For example: Say you want to cancel your pizza order from before it was made. Then the application performs a DELETE request for that specific order identifier, and the server deletes that instance from the system. Thus, the correct use of: GET, POST, PUT, and DELETE here more clearly distinguishes between a very well-built API and a poorly constructed feature. Using methods, the needed action is clearly indicated to the server, which makes the API very straightforward to understand.

Question 3:Imagine you are part of a team responsible for integrating different systems using SOAP. Can you provide a scenario where XML plays a crucial role in exchanging structured data between these systems, enabling seamless communication?

Let's continue with this example, assuming that Royal Bank of Canada (RBC) is our bank. Obviously, Royal Bank of Canada (RBC) has a payment system, but that system needs to communicate with Equifax to get the relevant credit report and report to Canada Revenue Agency (CRA) for tax purposes. The solutions for these three needs were built at different times, by different teams, using different programming languages. Getting any two of them to communicate reliably is a considerable challenge. This is precisely the type of problem for which eXtensible Markup Language (XML) and SOAP provide the best tools.

SOAP stands for Simple Object Access Protocol (SOAP), and it is a standard for exchanging information between distributed components over the Internet. If you need a reference point, XML looks a lot like HTML. It has opening and closing tags around the data. However, XML is used to describe information, not to format it for a web page as HTML does. When Royal Bank of Canada (RBC) sends a customer's information to Equifax for credit checks before approving the loan, the payment system creates the SOAP message and sends it to Equifax. Usually, this has two parts, the envelope which contains information about the message, such as the sender and potential security credentials, and the body where the customer data resides. When Equifax receives the SOAP message, it parses the XML, pulls out the informa tion, does the credit check, and sends the result back in another SOAP message. This is one area where eXtensible Markup Language (XML) really shines, representing complex and hierarchical data, while keeping it intact and structured. Even though two systems like Royal Bank of Canada (RBC) and Equifax might be entirely different in numerous ways, the XML representation bridges this gap-the message arrives intact every time. Moreover, SOAP has built-in security options, such as message encryption and signing. Thus, the message's content is protected during transportation and is readable only by the intended recipient. Such security is not optional but mandatory for a bank.

Another thing I like about SOAP is that it does not simply push the message over the wire and consider its job done. SOAP messages can be delivered with optional acknowledgements. If some error occurs, the system will know about it and can indicate exactly what went wrong. For credit checks or Canada Revenue Agency (CRA) tax submissions, data cannot get lost and a message cannot be sent twice by accident. These are all things that SOAP takes care of and thus, SOAP is mostly used in financial and healthcare-related systems.

Question 4: As an IoT enthusiast, you are designing a smart home automation system that interacts with various devices. How would you showcase the significance of APIs in enabling seamless communication and interoperability among different IoT devices within this scenario?

Imagine a house with a smart, Google made thermostat; automated Philips lighting, phone controllable Yale door locks, an Alexa-powered counter, and security cameras from Ring. Each of those things came from a different company, so how in the world do they all work together in a single system? APIs.

The course notes also use a waiter to explain the role of the API, which is quite effective. All a waiter does is take orders to the kitchen and bring back the prepared meals; he is not responsible for cooking. An API functions similarly. In an API, the requests and responses move between two systems. There is an API for each device in a smart home since the API functions as an intermediary.

So when I say to Alexa "turn off the lights," Alexa does not control the lights. Instead, it calls the Philips Hue API, which then sends the command to the bulb to turn off, and the bulb sends a confirmation back that it did so. In terms of the security system, if the Ring camera detects motion at the front door, its API can trigger both a check with the lock to see if it is locked and a command to the camera to start recording. This is not some kind of magic; it is simply an API calling another API.

As per the course notes, Internet of Things (IoT) is generally described as "the network of interconnected physical objects or 'things' that are embedded with sensors, software, and other technologies to connect and exchange data with other devices and systems over the Internet." So, while the thermostat, camera, and lights are definitely IoT devices, they are simply connected to the Internet and cannot work with other devices within the home on their own. However, thanks to APIs, data is shared, and devices respond to each other. APIs create a unique data-sharing environment where each device can respond accordingly. In other words, an API allows the thermostat to tell the lights to dim and the camera to alert the lock.

Without APIs, a "smart home" is just a bunch of expensive devices that don't communicate. Once connected through APIs, these devices form one system that will respond to you, protect your home, and save energy without your manual and constant input. That is how APIs become an essential element in the IoT universe and not just one of its features.

References

Algonquin College. (2022). Application Programming Interface (API). CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Algonquin College. (2022). API architectures. CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Algonquin College. (2022). API patterns or principles. CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Algonquin College. (2022). GraphQL. CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Algonquin College. (2022). REST API. CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Algonquin College. (2022). SOAP API. CST8916 – Remote Data and Real-Time Applications, Module 1. Brightspace Learning Management System.

Fielding, R. T. (2000). Architectural styles and the design of network-based software architectures [Doctoral dissertation, University of California, Irvine]. https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

In conclusion, the module provides an overview of the key concepts and terms related to web services-based API architectures and IoT technology. It explains the importance of APIs in building software applications and the different types of APIs available. It also examines the different types of web services-based API architectures and their advantages and disadvantages. Overall, the module provides a solid foundation for understanding web services-based API architectures and their applications in real-world scenarios.

References

- "Asynchronous Python for the Complete Beginner - YouTube" (2020, February 3). [Video]. YouTube. **https://www.youtube.com/watch?v=lsMQRaeKNDk (https://www.youtube.com/watch?v=lsMQRaeKNDk)**
- "Building a Python REST API with FastAPI and Docker" (2020, May 28). [Video]. YouTube. **https://www.youtube.com/watch?v=eIQh02xuVw4 (https://www.youtube.com/watch?v=eIQh02xuVw4)**
- "GraphQL vs REST: Overview" by Sturgeon, P. (2017). Retrieved from **https://philsturgeon.uk/api/2017/01/24/graphql-vs-rest-overview (https://philsturgeon.uk/api/2017/01/24/graphql-vs-rest-overview)**
- "HTTP" (n.d.). MDN Web Docs. Retrieved from **https://developer.mozilla.org/en-US/docs/Web/HTTP (https://developer.mozilla.org/en-US/docs/Web/HTTP)**
- "HTTP methods" (n.d.). MDN Web Docs. Retrieved from **https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods (https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)**
- "Implementing a GraphQL server" (n.d.). HowToGraphQL. Retrieved from **https://www.howtographql.com/graphql-js/0-introduction/ (https://www.howtographql.com/graphql-js/0-introduction/)**
- "Official GraphQL documentation" (n.d.). GraphQL. Retrieved from **https://graphql.org/ (https://graphql.org/)**
- "Representational State Transfer (REST)" (n.d.). IBM Knowledge Center. Retrieved from **https://www.ibm.com/docs/en/api-connect/10.0.x?topic=transfer-representational-state-transfer-rest (https://www.ibm.com/docs/en/api-connect/10.0.x?topic=transfer-representational-state-transfer-rest)**
- "REST API Tutorial" (n.d.). RESTful API. Retrieved from **https://restfulapi.net/ (https://restfulapi.net/)**
- "Building a RESTful API with Node.js and Express" by Scotch.io. Retrieved from **https://scotch.io/tutorials/build-a-restful-api-using-node-and-express-4**

(https://scotch.io/tutorials/build-a-restful-api-using-node-and-express-4)

- "Architectural styles and the design of network-based software architectures" by Fielding, R. T. (2000). (Doctoral dissertation, University of California, Irvine). Retrieved from **https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm (https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm)**

- "HTTP/1.1 Semantics and Content" (RFC 7231) by Fielding, R., Nottingham, M., & Reschke, J. (2014). Internet Engineering Task Force (IETF). Retrieved from **https://tools.ietf.org/html/rfc7231 (https://tools.ietf.org/html/rfc7231)**

- "Web Scraping Indeed for Key Data Science Job Skills - Python Tutorial - YouTube" (2019, February 8). [Video]. YouTube. **https://www.youtube.com/watch?v=ByGJQzlzxQg (https://www.youtube.com/watch?v=ByGJQzlzxQg)**

- "5 Minute Python Scripts: Schedule Web Scrapers with Cron - YouTube" (2017, February 22). [Video]. YouTube. **https://www.youtube.com/watch?v=sTGgBoFBDAY (https://www.youtube.com/watch?v=sTGgBoFBDAY)**

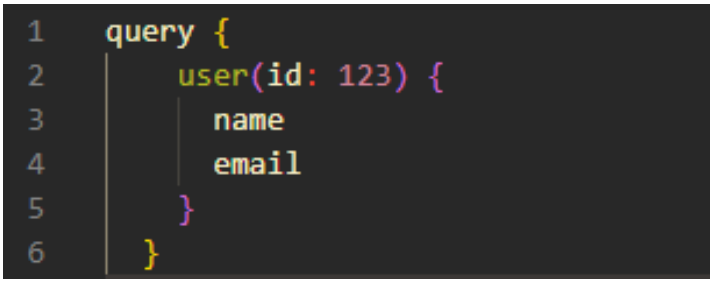
GraphQL

GraphQL is an open-source query language and runtime for APIs (Application Programming Interfaces). It was developed by Facebook in 2012 and later released to the public in 2015. GraphQL is designed to provide a more efficient and flexible approach to fetching data from servers compared to traditional RESTful APIs. It allows clients to request specific data and shape the response according to their needs. Let's take a look at how it works and some of its advantages

How does it work?

GraphQL is a query language that allows clients to specify the data they need, and the server returns only that data. This is done by defining a schema, which describes the types of data available and the relationships between them. Clients then use this schema to request the data they need. Here's an example of a simple GraphQL query.

Example



In this query, we're asking for the name and email of the user with the ID of 123. The server will only return those fields, and nothing more.

Additionally, GraphQL supports mutations, which allow clients to modify data on the server. Here's an example of a mutation.

Example



In this mutation, we're updating the name and email of the user with the ID of 123. The server will return the updated user object.

Advantages of using GraphQL

1. **Reduced over-fetching and under-fetching:**

With REST APIs, clients often have to make multiple requests to retrieve all the data they need. This can result in over-fetching (retrieving more data than needed) or under-fetching (not retrieving enough data). With GraphQL, clients can request only the data they need, and nothing more.

2. **Strong typing:**

GraphQL uses a schema to define the types of data available and the relationships between them. This allows clients to know exactly what data they can request and what shape it will be in.

3. **Self-documenting APIs:**

Because GraphQL schemas define the types of data available and the relationships between them, they can be used to automatically generate documentation for the API.

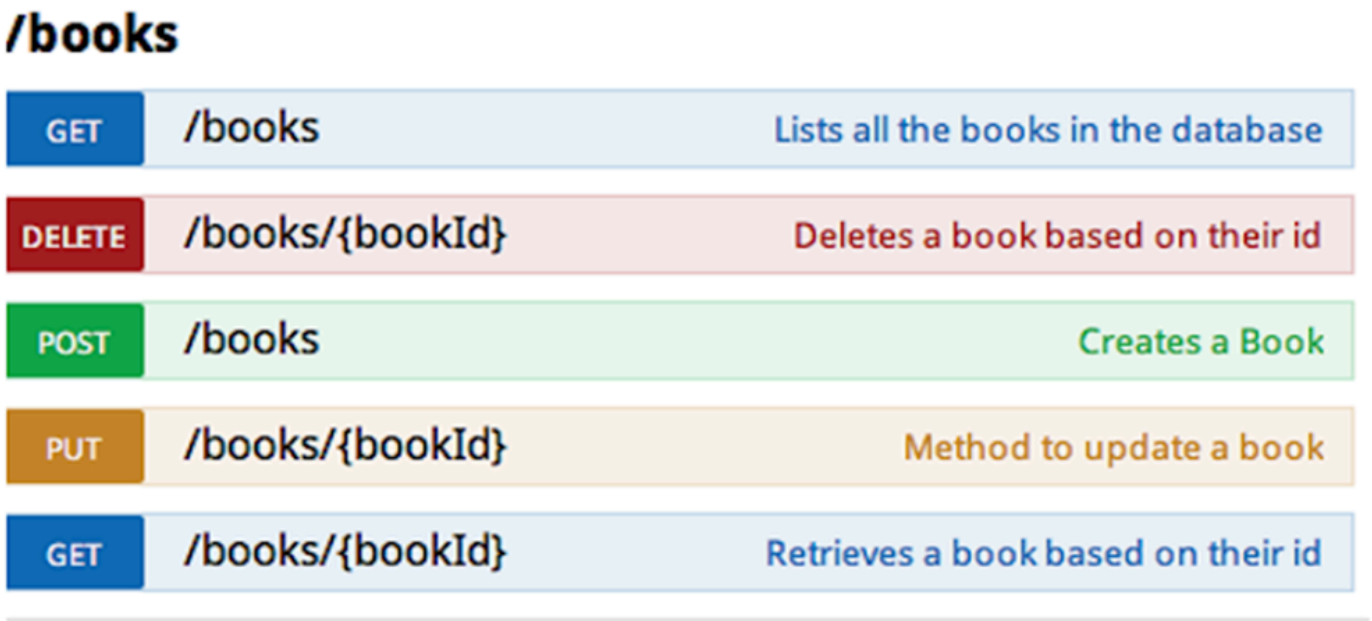
4. **Increased developer productivity:**

With GraphQL, clients can easily discover the available data and relationships, and can modify their queries and mutations without needing to change the server.

© Algonquin College 2022

REST API

REST (Representational State Transfer) is a widely used architectural style for designing web services, and RESTful APIs (Application Programming Interfaces) are a popular implementation of the REST architecture. RESTful APIs provide a standardized way for different systems to communicate with each other over HTTP (Hypertext Transfer Protocol) using a set of simple, well-defined operations. Illustrated below is this operation.



Let's dive a little deeper into RESTful APIs by examining how they work, and how to build and consume them.

What is a RESTful API?

A RESTful API is a web-based API that uses HTTP methods (GET, POST, PUT, DELETE) to create, read, update, and delete resources (data objects) on a server. RESTful APIs use a standardized set of HTTP operations and response codes to provide a simple, language-agnostic interface for different systems to communicate with each other.

RESTful APIs are based on the principles of the REST architectural style, which emphasizes the importance of resources, hypermedia, and statelessness. RESTful APIs use URIs (Uniform Resource Identifiers) to identify resources, and they return resource representations in a standard format such as JSON (JavaScript Object Notation) or XML (Extensible Markup Language).

How do RESTful APIs work?

RESTful APIs work by defining a set of resources on a server and exposing them through a set of HTTP operations. Each resource is identified by a unique URI, and the client can perform different operations on the resource by sending HTTP requests to the server.

For example, let's say we have a RESTful API for a bookstore that exposes a set of books as resources. The URI for a book resource might be /books/{id}, where {id} is the unique identifier of the book. To retrieve the details of a book, the client can send an HTTP GET request to the URI /books/{id}. To add a new book, the client can send an HTTP POST request to the URI /books, along with the details of the new book in the request body.

RESTful APIs also use HTTP response codes to provide feedback to the client. For example, if the client sends an HTTP GET request to retrieve the details of a book that does not exist, the server might respond with an HTTP 404 Not Found error code. If the client sends an HTTP POST request to add a new book successfully, the server might respond with an HTTP 201 Created status code.

The Principles of REST API

RESTful APIs are designed around a few key principles, including:

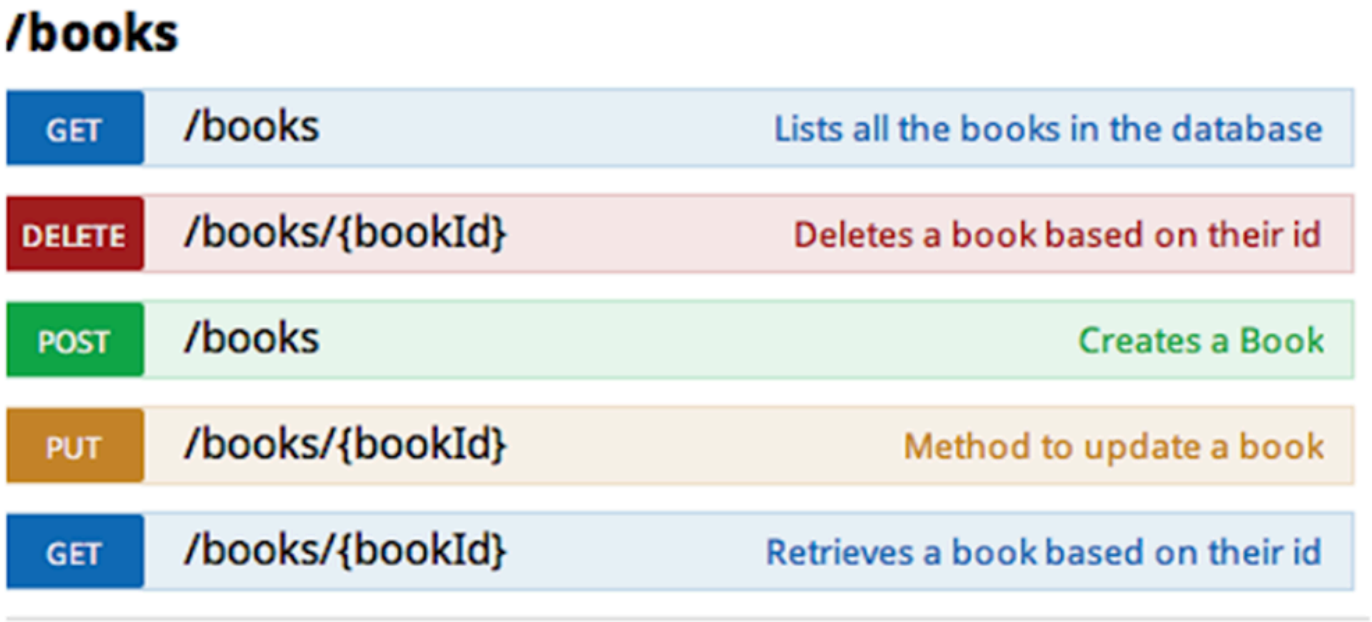
1. Uniform interface: The API should have a consistent interface that is independent of the client or server.
2. Client-server architecture: The API should separate the concerns of the client and server.
3. Statelessness: The API should not retain any state about the client or server.
4. Cacheability: The API should be designed to be cacheable to improve performance.
5. Layered system: The API should be designed to be layered to improve scalability and flexibility.

HTTP Methods and Status Codes in REST API

RESTful APIs use HTTP methods to define the actions that can be performed on resources. The most commonly used HTTP methods are:

1. GET: Used to retrieve a resource from the server.
2. POST: Used to create a new resource on the server.
3. PUT: Used to update an existing resource on the server.
4. DELETE: Used to delete a resource from the server.

The figure below illustrates the HTTP methods available for the RESTful API of the library's book.



HTTP status codes are used to indicate the status of the request. Some common HTTP status codes are:

1. 200 OK: The request was successful.
2. 201 Created: The resource was successfully created.
3. 400 Bad Request: The request was malformed or invalid.
4. 401 Unauthorized: The client is not authorized to access the resource.
5. 404 Not Found: The requested resource was not found.
6. 500 Internal Server Error: An error occurred on the server.

EXAMPLE OF A REST API

Let's say we want to create a RESTful API to manage a list of users. Here is an example of what the API might look like:

1. Resource: User
2. URI: /users
3. HTTP method: GET
4. Headers: None
5. Request body: None
6. Response body: Returns a list of all users

© Algonquin College 2022

SOAP API

SOAP (Simple Object Access Protocol) is a messaging protocol that is used to exchange structured data between applications over the internet. It is an XML-based protocol that uses an envelope format to define how data should be exchanged. SOAP is often used in enterprise applications where the reliability and security of the data being exchanged are critical.

SOAP is designed to work with many different types of data and transport protocols, which makes it a flexible choice for a wide range of applications. The SOAP protocol is based on the idea of sending a message to a remote service, which will then process the message and send a response. The message can contain a wide range of information, such as requests, responses, and errors. The SOAP protocol uses an XML-based messaging format to encode this information, making it easy to parse and understand.

SOAP Structure

SOAP messages are made up of two parts:

1. The envelope; and
2. The body.

The envelope contains information about the message, such as the sender and receiver, and any security information that may be required, while the body contains the actual data being exchanged between the applications. SOAP messages can also contain header elements, which can be used to provide additional information about the message or the data being exchanged.

Advantages of using SOAP

One of the key advantages of SOAP is its flexibility. Because SOAP is based on XML, it can be used to exchange data between applications written in any programming language. This makes it an ideal choice for enterprise applications that may be using a variety of different technologies. Additionally, SOAP can be used over a variety of different transport protocols, including HTTP, SMTP, and TCP/IP.

Another advantage of SOAP is its built-in support for security and reliability. SOAP messages can be encrypted and signed, which ensures that they can only be read by the intended recipient. Additionally, SOAP includes built-in support for message delivery confirmation and error handling, which helps ensure that data is delivered reliably and that errors are handled gracefully. Below is an example of the SOAP message.

```xml
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">
    <SOAP-ENV:Header>
        <AuthenticationHeader>
            <Username>johnsmith</Username>
            <Password>password</Password>
        </AuthenticationHeader>
    </SOAP-ENV:Header>
    <SOAP-ENV:Body>
        <GetStockPrice>
            <Symbol>ABC</Symbol>
        </GetStockPrice>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

In this example, the SOAP message is requesting the stock price for a company with the symbol "ABC". The message includes an authentication header, which contains the username and password of the user making the request. The message is sent to a remote service, which will then process the message and send a response containing the requested stock price.

In conclusion, SOAP is a messaging protocol that is used to exchange structured data between applications over the internet. It is an XML-based protocol that uses an envelope format to define how data should be exchanged. SOAP is designed to be flexible and work with many different types of data and transport protocols. Additionally, SOAP includes built-in support for security and reliability, making it an ideal choice for enterprise applications where the reliability and security of the data being exchanged are critical.

© Algonquin College 2022

API Patterns or Principles

In addition to the specific architectures, web services-based APIs also follow certain patterns or principles as to ensure consistency and interoperability. These patterns include:

1. **Service-Oriented Architecture (SOA):**

SOA is an architectural approach that focuses on building services that can be reused across different applications. SOA services are typically independent and modular, and can be combined to create more complex applications.

Example

Let's say you work for a large retail company that has different systems for managing inventory, shipping, and customer orders. In an SOA approach, you would create separate services for each of these functions, which can be reused across different applications. For example, the inventory service could be used by the shipping system to determine which products are available for shipping, and by the order system to prevent customers from ordering out-of-stock items.

2. Micro services:

Micro services is an architectural style that involves breaking down a monolithic application into smaller, independent services that can be developed and deployed separately. Each micro service has its own API, and these APIs work together to provide the functionality of the overall application.

Example

Imagine you work for a food delivery app that allows customers to order from different restaurants. In a monolithic architecture, all the code for the app would be in one large system. However, with micro services, you could break down the app into smaller, independent services. For example, you might have a service for managing user accounts, a service for restaurant listings, and a service for processing orders. Each of these services would have its own API, and the app would work by calling these APIs as needed.

3. **Resource-Oriented Architecture (ROA):**

ROA is an architectural style that focuses on defining resources and their relationships in a networked environment. It emphasizes the use of standard HTTP verbs and URIs to manipulate resources, and is often used in RESTful APIs.

Example

Let's say you are building a weather app that allows users to get current weather conditions for different locations. In a ROA approach, you would define each location as a resource, with its own unique URI. To get the weather for a specific location, the app would send an HTTP GET request to the corresponding URI. For example, the URI for the weather in New York City might be **http://weatherapp.com/locations/nyc (http://weatherapp.com/locations/nyc)**. The app could also use standard HTTP verbs like PUT and DELETE to modify or delete resources.

Overall, web services-based API architectures provide a standardized and interoperable way to build APIs that can be used across different systems and platforms. By following established patterns and principles, developers can ensure that their APIs are well-designed, scalable, and easy to use.

For the remainder of the module, we are going to break each of the architectural styles down even further, starting with SOAP API.

© Algonquin College 2022

API architectures

Web services-based API architectures are a set of principles, protocols, and technologies used to build Application Programming Interfaces (APIs) that can be accessed over the internet. These architectures provide a standardized approach for developers to build APIs that can be used by different systems, devices, and applications.

The most common web services-based API architectures include:

1. SOAP (Simple Object Access Protocol):

SOAP is a messaging protocol used to exchange structured information between systems over the internet. It uses Extensible Markup Language (XML) as its message format and can be transported using a variety of protocols, such as HTTP, SMTP, or TCP. To learn more about SOAP Web Services, as well as its specifications and components, please watch the following video: **https://www.youtube.com/watch?v=sTGgBoFBDAY (https://www.youtube.com/watch?v=sTGgBoFBDAY)**

2. REST (Representational State Transfer):

REST is an architectural style that uses HTTP as its underlying protocol for communication. RESTful APIs use a set of standard HTTP methods, such as GET, POST, PUT, and DELETE, to access and manipulate resources. Data is typically returned in formats such as JSON or XML. For more on REST API, please watch the following introductory video: **https://www.youtube.com/watch?v=lsMQRaeKNDk (https://www.youtube.com/watch?v=lsMQRaeKNDk)**

3. GraphQL:

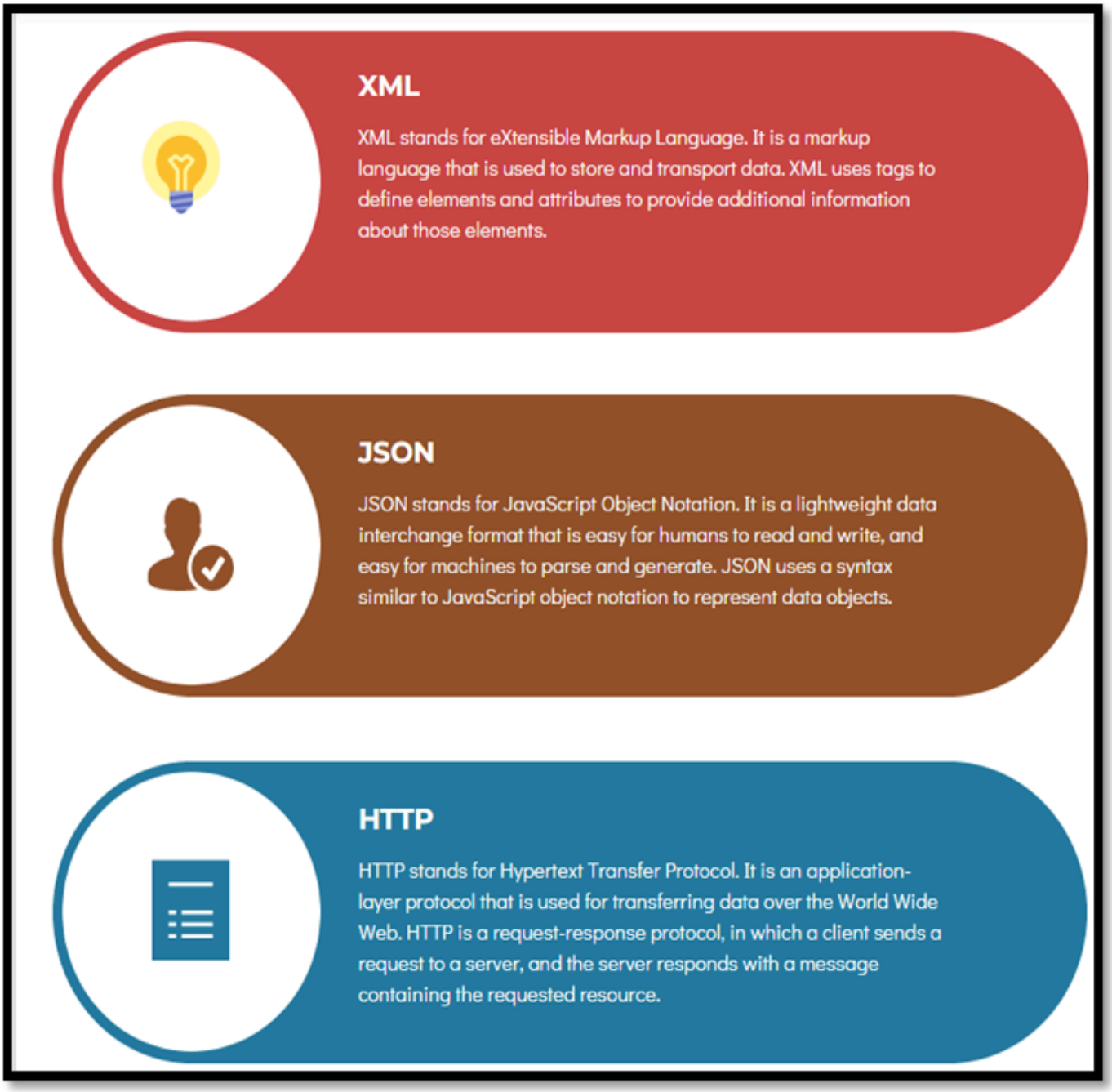
GraphQL is a query language and runtime for APIs that was developed by Facebook. It provides a single endpoint for querying data, allowing clients to request only the data they need. GraphQL APIs are defined using a schema, which describes the available data and operations. See what the following video has to say about GraphQL: **https://www.youtube.com/watch?v=eIQh02xuVw4 (https://www.youtube.com/watch?v=eIQh02xuVw4)**

SUPPORTING DEFINITIONS

XML: XML stands for eXtensible Markup Language. It is a markup language that is used to store and transport data. XML uses tags to define elements and attributes to provide additional information about those elements.

JSON: JSON stands for JavaScript Object Notation. It is a lightweight data interchange format that is easy for humans to read and write, and easy for machines to parse and generate. JSON uses a syntax similar to JavaScript object notation to represent data objects.

HTTP: HTTP stands for Hypertext Transfer Protocol. It is an application-layer protocol that is used for transferring data over the World Wide Web. HTTP is a request-response protocol, in which a client sends a request to a server, and the server responds with a message containing the requested resource.



These web services-based API architectures have different strengths and weaknesses, and choosing the right one depends on the specific needs of the application being developed. SOAP, for example, provides strong security and transactional support, making it a good choice for enterprise-level applications. REST, on the other hand, is more lightweight and easier to implement, making it a popular choice for web and mobile applications.

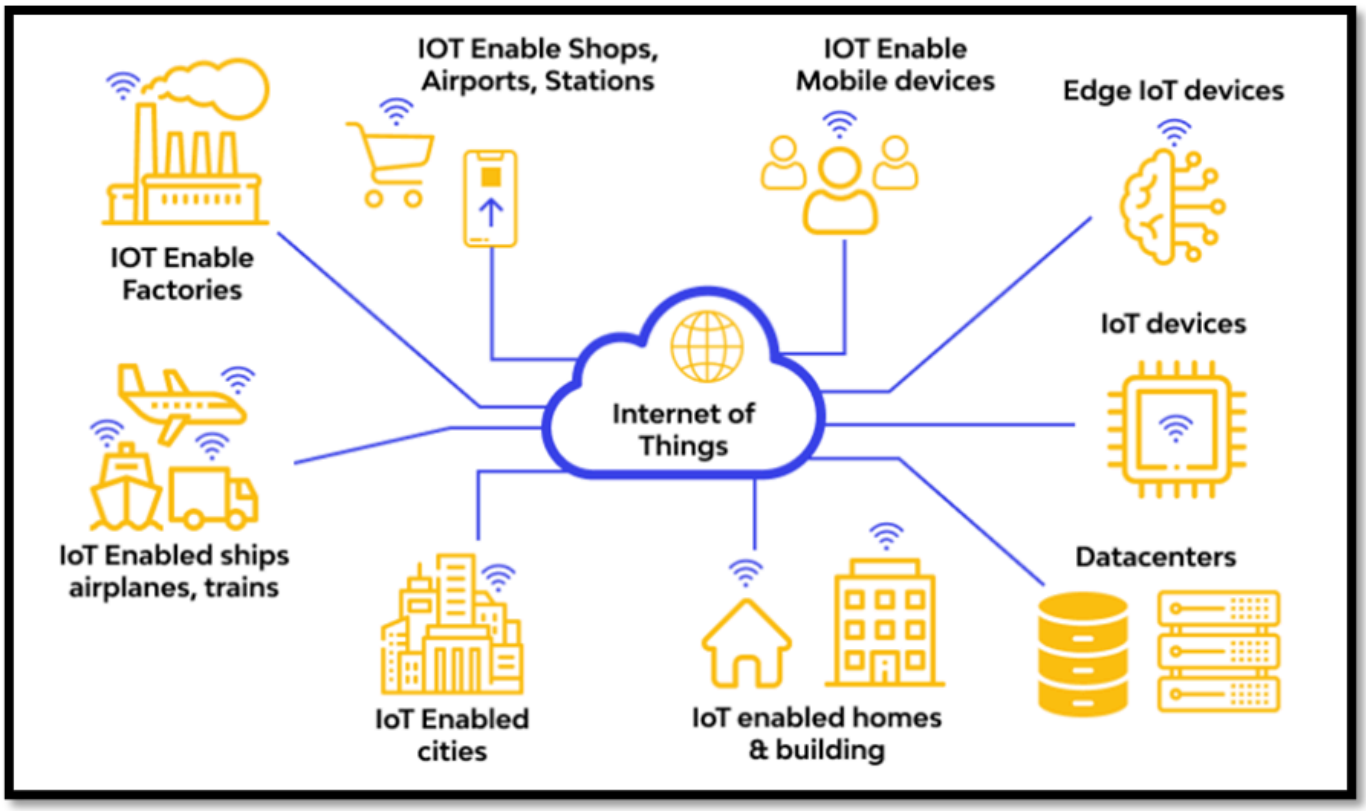
© Algonquin College 2022

Application Programming Interface (API)

With that detailed introduction under your belt, let's begin our deep dive into Application Programming Interface (API). An API is like a waiter at a restaurant. Just like a waiter takes your order, delivers it to the kitchen, and brings your food back to your table, an API helps different software applications communicate with each other. Another example is when you use a social media app. The app might use an API to request information from a server, such as the list of your friends or the posts on your newsfeed. The API retrieves this information and sends it back to the app, which then displays it to you.

Ultimately, an API helps one application request information or functionality from another application, and then delivers the response back to the first application.

The illustration below is an excellent depiction of its process.



APIs can be used in many different ways, from sharing data between applications to accessing functionality and services from other software systems. They're an important tool for building software applications that can interact with each other and provide a seamless user experience.

IOT and API

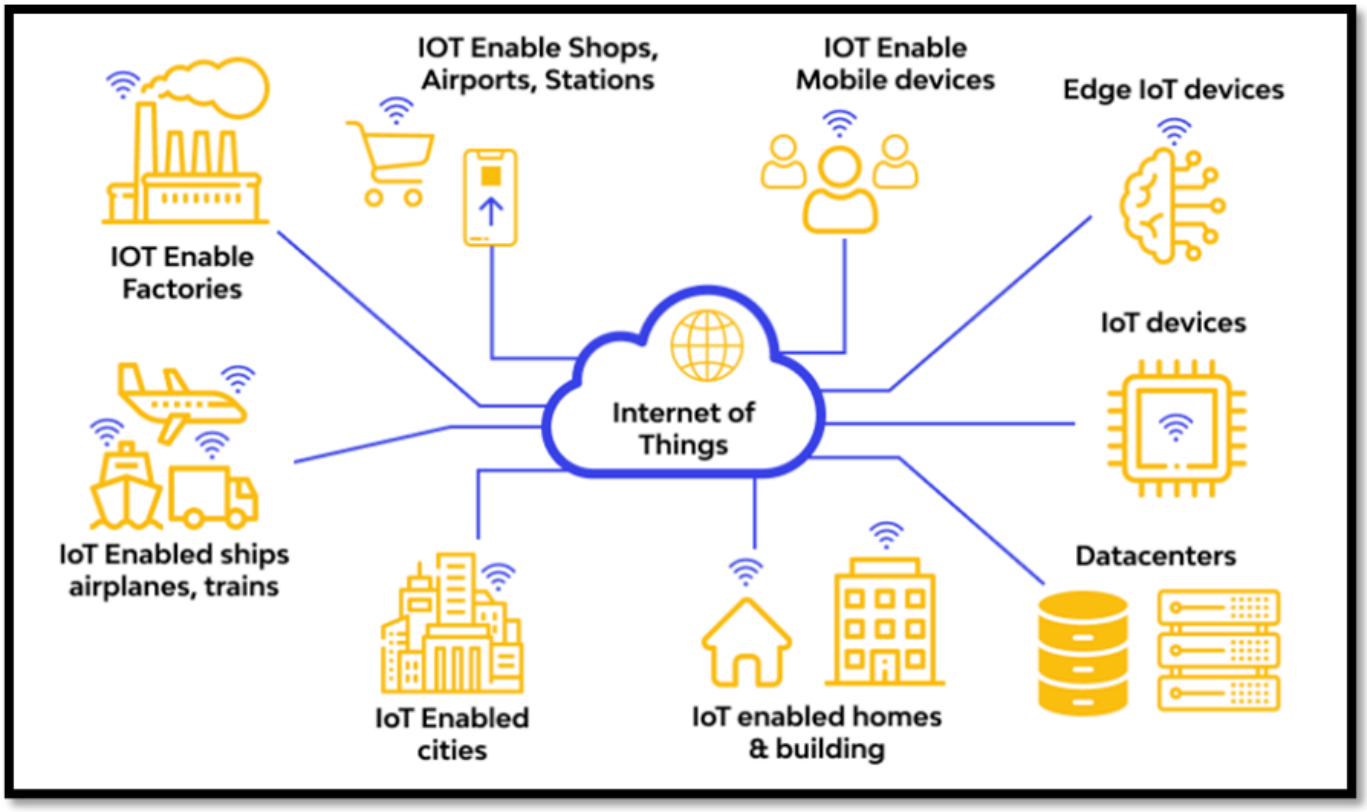
When it comes to IoT, an API can be thought of as a bridge between different devices and systems, allowing them to communicate with each other and share data. For example, let's say you have a smart thermostat that is connected to the internet. The thermostat may have an API that allows other devices, such as a smart home assistant, to access its data and functionality.

Using the API, the smart home assistant can send requests to the thermostat to change the temperature or check the current temperature in the room. The thermostat then responds with the requested data or performs the requested action.

Similarly, other devices in your home, such as smart lights, sensors, and appliances, can also have APIs that allow them to communicate with each other and share data. This can enable a wide range of applications, such as automated home security systems, energy-efficient home automation, and remote monitoring of home appliances.

Overall, APIs are a crucial component of IoT systems, as they enable different devices and systems to communicate and work together, creating a seamless and connected experience for users.

Here's an example of how IoT and API work together.



© Algonquin College 2022

API Architectures

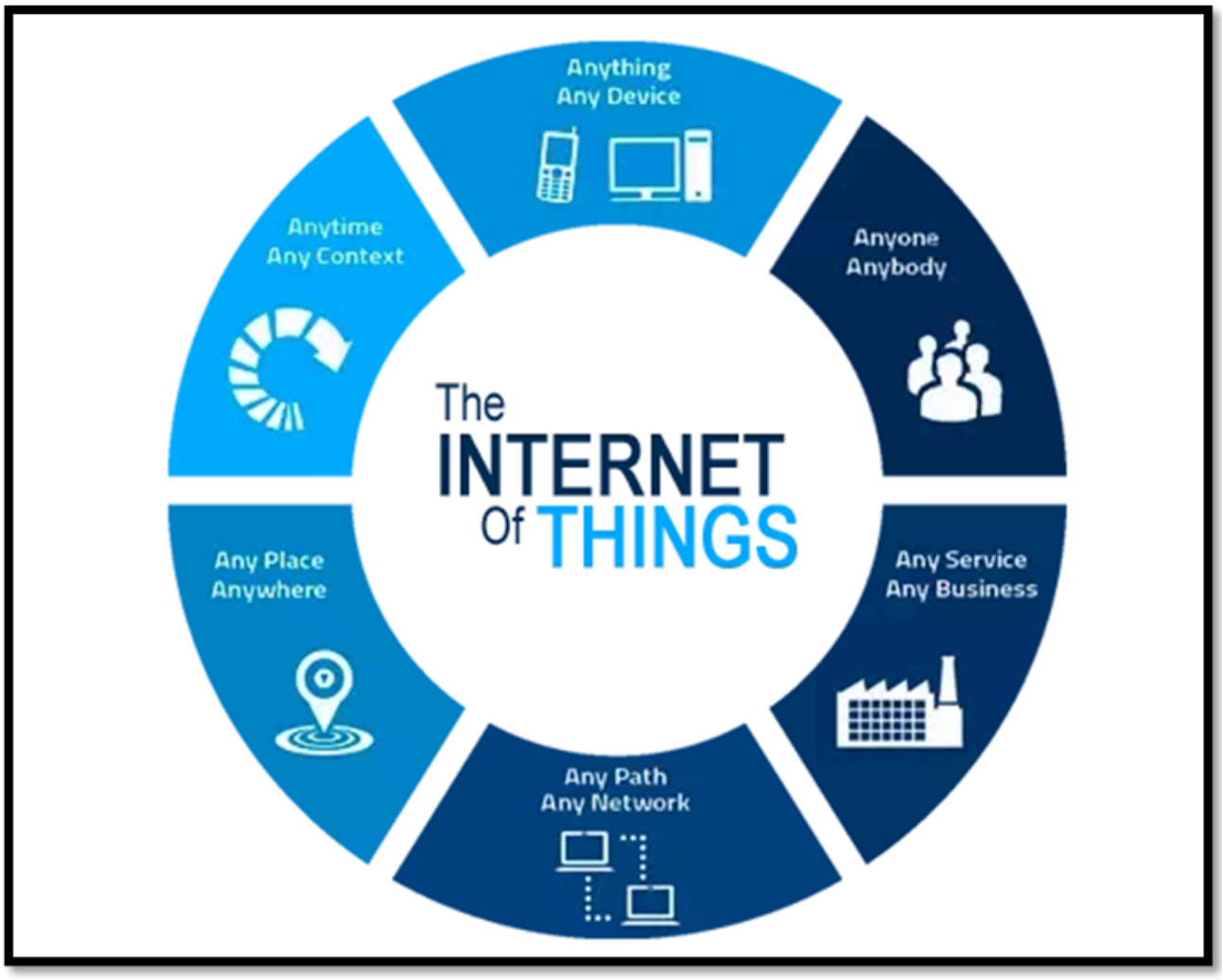
As you begin this first module, a few important concepts are necessary to help secure foundational knowledge. Let's examine Web Services, focusing more specifically on API and IOT, and then begin our deep dive into API Architecture. An API, or Application Programming Interface, is a set of protocols, routines, and tools for building software applications. APIs allow different applications to communicate with each other, exchange data, and access each other's functionality. APIs define how different software components should interact, which helps to enable data sharing, communication, and functionality between different systems.

APIs come in many different forms, from simple libraries that provide access to specific functions or data structures, to complex web services that allow developers to access data and functionality over the internet. APIs can be used to build a wide range of applications, including web and mobile apps, IoT devices, and enterprise software systems.

Watch the following video as a great introduction into APIs.

- **https://www.youtube.com/watch?v=ByGJQzlzxQg (https://www.youtube.com/watch?v=ByGJQzlzxQg)**

Furthermore, IoT, or Internet of Things is an equally important concept to understand, as It refers to a network of physical devices, vehicles, home appliances, and other items that are embedded with sensors, software, and connectivity, enabling them to connect and exchange data over the internet. The following figure shows how the IoT connect everything in the world



IoT devices can range from simple consumer products, like smart thermostats and home assistants, to complex industrial systems, just like factory machinery and supply chain management systems. These devices are connected to the internet, allowing them to communicate with each other and with other software systems.

IoT technology enables a wide range of applications, such as smart homes, smart cities, and industrial automation. For example, in a smart home, a thermostat might communicate with a smart lightbulb to automatically adjust the lighting based on the temperature in the room. In an industrial setting, sensors might be used to monitor the performance of machines and alert maintenance teams to potential problems before they occur.

IoT technology is rapidly growing and evolving, with new devices and applications being developed all the time. It has the potential to revolutionize the way we live and work, making our lives more convenient, efficient, and connected.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain API and IOT

02 Summarize web services-based API architectures

03 Differentiate between APIs: REST, SOAP, GraphQL

04 Interpret API architectural patterns

Key Terms and Concepts

1. API: Application Programming Interface
2. IoT: Internet of Things
3. REST: Representational State Transfer
4. SOAP: Simple Object Access Protocol
5. GraphQL: Graph Query Language
6. XML: eXtensible Markup Language
7. JSON: JavaScript Object Notation
8. HTTP: Hypertext Transfer Protocol

Implementation Challenges

Implementing web services-based API architectural patterns can be challenging, and there are several factors that need to be considered. These include:

1. **Compatibility with legacy systems:**

Many organizations have legacy systems that need to be integrated with modern applications. It is important to ensure that the chosen architecture pattern is compatible with these legacy systems.

2. **Scalability:**

Applications need to be able to handle large amounts of traffic, and the chosen architecture pattern needs to be scalable to meet the demand.

3. **Security:**

Security is a critical consideration when designing web services-based API architectural patterns. It is important to ensure that the architecture pattern provides the necessary security features, such as encryption and authentication.

4. **Developer experience:**

The chosen architecture pattern should be easy for developers to work with and should provide the necessary tools and resources to build and deploy applications.

© Algonquin College 2022

IOT and API architectures

IoT devices are typically small, low-power devices that are designed to connect to the internet and exchange data with other devices or applications. They may have limited processing power, memory, and battery life, which makes it important to choose an API architecture that is lightweight, efficient, and scalable.

One of the most popular API architectures for IoT devices is the RESTful API. RESTful APIs use the HTTP protocol to exchange data between clients and servers, and they are designed to be lightweight, simple, and scalable. They use standard HTTP methods like GET, POST, PUT, and DELETE to perform CRUD (Create, Read, Update, and Delete) operations on resources, and they use resource URIs to identify and access those resources.

In addition to RESTful APIs there are other API architectures that may be suitable for IoT devices, depending on the specific use case. For example, some IoT devices may require a more complex API architecture that supports advanced features like authentication, encryption, and message routing. In these cases, a SOAP (Simple Object Access Protocol) API or a custom API architecture may be more appropriate.

When selecting an API architecture for IoT devices, it is important to consider factors like scalability, security, and interoperability. Scalability is important because IoT applications may involve thousands or even millions of devices, and the API architecture must be able to handle this level of traffic and data exchange. Security is important because IoT devices may be used to collect sensitive data, and the API architecture must be designed to protect that data from unauthorized access or tampering. Interoperability is important because IoT devices may need to communicate with other devices or applications that use different protocols or APIs.

While both can be used for IoT devices, RESTful APIs are often considered the better choice for several reasons. These reasons include:

1. **Lightweight and Flexible:**

One of the main advantages of RESTful APIs is that they are lightweight and flexible. Unlike SOAP, which relies on a strict message format and a complex set of rules for transmitting data, RESTful APIs allow developers to structure requests and responses in a variety of ways. This makes them ideal for IoT devices, which often have limited processing power and storage capacity.

2. **Scalable:**

RESTful APIs are designed to be scalable, meaning that they can handle many requests from multiple devices simultaneously. This is essential for IoT devices, which often need to communicate with each other and with the cloud in real-time.

3. **Resource-Oriented:**

Another advantage of RESTful APIs is that they are resource-oriented. This means that they are designed to represent resources, such as devices, sensors, and data streams, as URLs. This makes it easy for developers to create and manage IoT APIs, and it simplifies the process of retrieving and updating data from IoT devices.

4. **Security:**

RESTful APIs offer a range of security options, including HTTPS and OAuth2. This is essential for IoT devices, which often need to transmit sensitive data over the internet.

5. **Ease of Use:**

Finally, RESTful APIs are generally considered to be easier to use than SOAP-based APIs. They require less code and are easier to understand, which makes them ideal for developers who are new to IoT development.

Example

Consider a smart home system that uses IoT devices to control various appliances and monitor energy usage. A RESTful API could be used to manage the devices and retrieve data from them, such as temperature readings and power usage statistics. The API could be designed with resource-oriented URLs, making it easy for developers to access and manipulate data from the devices. Security could be ensured by using HTTPS, and the lightweight and flexible nature of RESTful APIs would make it easy to manage a large number of devices simultaneously.

Here's an example of a RESTful API for an IoT-based weather monitoring system:

- Endpoint: GET /devices
- Endpoint: GET /devices/{deviceId}
- Endpoint: GET /devices/{deviceId}/temperature
- Endpoint: GET /devices/{deviceId}/humidity
- Endpoint: GET /devices/{deviceId}/pressure
- Endpoint: POST /devices/{deviceId}/alert
- Endpoint: POST /devices/{deviceId}/control

Overall, while SOAP-based APIs have their advantages in certain contexts, RESTful APIs are generally considered the better choice for IoT devices. Their lightweight, flexible, and scalable nature, combined with their ease of use and security features, make them ideal for managing and interacting with IoT devices in a wide range of contexts.

© Algonquin College 2022

Web Services-Based API Architectural Patterns

The most used web services-based API architectural patterns are Representational State Transfer (REST) and Simple Object Access Protocol (SOAP). Both have their unique advantages and disadvantages, and their suitability depends on the use case. Assessing the Suitability of Different Web Services-based API Architectural Patterns means learning the difference between REST and SOAP. Let's quickly differentiate between them before we continue.

RESTful APIs

RESTful APIs are ideal for web applications that require fast and efficient data retrieval. RESTful APIs use HTTP requests to retrieve and manipulate data, and they are lightweight and scalable. Furthermore, RESTful APIs are also stateless, which means that they do not store any client session information, making them easier to scale. They are suitable for applications that require data to be transferred quickly, such as social media, e-commerce, and news sites.

SOAP

On the other hand, SOAP APIs are ideal for applications that require a high level of security, reliability, and transactional support. SOAP APIs use XML to send and receive data and rely on a messaging protocol to ensure the integrity and security of data transmission. They are suitable for applications that require complex transactions, such as financial services, healthcare, and enterprise applications.

Criteria	**REST**	**SOAP**
Full form	Representational State Transfer	Simple Object Access Protocol
Type	REST is an architectural style.	SOAP is a protocol.
Design	REST design is resource-based, using HTTP verbs for communication.	SOAP design is message-based, using XML for communication.
Data formats	REST uses multiple data formats, including JSON, XML, and plain text.	SOAP only uses XML for message exchange.

Security	REST is less secure compared to SOAP.	SOAP has built-in WS-Security for authentication, authorization, and encryption.
Transport	REST uses various protocols, including HTTP, HTTPS, and SSL.	SOAP typically uses only HTTP or HTTPS.
Caching	REST APIs are highly cacheable.	SOAP APIs are not cacheable by default.
Performance	REST APIs are faster and have lower latency.	SOAP APIs are slower and have higher latency due to the additional processing of XML.
Flexibility	REST APIs are flexible and adaptable.	SOAP APIs are less flexible and less adaptable.
State management	REST APIs use client-server statelessness.	SOAP APIs can use stateful or stateless communication.
Error handling	REST APIs use HTTP error codes for error handling.	SOAP APIs use their own error-handling mechanism, including the use of SOAP faults.
Scalability	REST APIs are more scalable than SOAP APIs.	SOAP APIs are less scalable due to their use of XML and complex processing.
Usage	REST APIs are suitable for web, mobile, and IoT applications.	SOAP APIs are suitable for enterprise-level applications, especially in complex, mission-critical systems.

Note: It's important to note that neither REST nor SOAP is inherently better than the other, as their suitability depends on the specific use case and requirements of the application.

Watch following video to learn more about the differences between REST, SOAP, as well as GraphQL: **https://www.youtube.com/watch?v=__2YQ2z8iac (https://www.youtube.com/watch?v=__2YQ2z8iac)**

Example

Additionally, using an example will help you better understand their differences. SOAP is like an envelope while REST is just a postcard. Certainly, a postcard is faster and cheaper to send than an envelope, but it could still be wrapped within something else, even an envelope.

You can just read a postcard too, while an envelope takes a few extra steps, like opening or unwrapping to access what's inside.



© Algonquin College 2022

GraphQL

GraphQL is a query language and runtime for APIs. It allows clients to request specific data from a server, rather than receiving the entire data set. While GraphQL has gained popularity in recent years, it may not always be the best choice for IoT devices. Here, we are going to discuss some of the reasons why it may not be a good idea to use GraphQL for IoT devices.

1. **Resource Constraints**

IoT devices often have limited resources, such as memory and processing power. GraphQL queries can be very complex and consume a lot of resources. This can cause performance issues and drain the device's battery. IoT devices need to be efficient and optimized for their specific use case, and using GraphQL may not be the best fit for this.

2. **Network Connectivity**

IoT devices often have intermittent or limited network connectivity. GraphQL relies heavily on network connectivity to retrieve data from a server. If the network connection is lost, the query will fail, and the device will not receive the data it needs. This can be especially problematic for critical applications, such as healthcare monitoring devices, where data needs to be received in real-time.

3. **Overhead**

GraphQL adds an additional layer of overhead to the communication between devices and servers. The GraphQL server must parse the query, validate it, and execute it. This adds additional latency to the request and can cause delays in data delivery. In IoT applications where real-time data is critical, the added latency could be detrimental.

4. **Security**

Another issue with GraphQL for IoT devices is security. GraphQL provides a powerful query language that can be used to access sensitive data. If not properly secured, this can result in security vulnerabilities that can be exploited by attackers. In contrast, RESTful APIs are typically simpler and easier to secure, making them a better choice for IoT devices.

© Algonquin College 2022

Architectures for Mobile Applications and IoT

Web services-based API architectural patterns are the backbone of modern software development. They allow different applications and services to communicate and share data seamlessly, irrespective of their underlying technology and implementation. However, selecting the right API architectural pattern is crucial for achieving the desired performance, scalability, and security of the system.

In this module, we will assess the suitability of different web services-based API architectural patterns, examine their strengths and weaknesses, and select the best pattern for various use-cases. We will also discuss the implementation challenges and considerations involved in designing and deploying these API patterns.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Critique the suitability of different web services-based API architectural patterns

02 Outline the best architecture pattern for various use-cases

03 Summarize the implementation challenges and considerations

Key Terms and Concepts

1. API: Application Programming Interface
2. IoT: Internet of Things
3. REST: Representational State Transfer
4. SOAP: Simple Object Access Protocol
5. GraphQL: Graph Query Language
6. XML: eXtensible Markup Language
7. JSON: JavaScript Object Notation

8. HTTP: Hypertext Transfer Protocol

Use-Cases

Selecting the Best Architecture Pattern for Various Use-Cases is important. It requires careful consideration of several factors, including scalability, security, reliability, transaction support, and ease of use. Let's look at some scenarios and determine the best architecture pattern for each one.

Scenario 1: E-Commerce

An e-commerce website that requires fast data retrieval and ease of use.
The best architecture pattern for this scenario is RESTful APIs. RESTful APIs are lightweight and scalable, making them ideal for applications that require fast data retrieval. RESTful APIs are also easy to use, which is important for an e-commerce website that needs to be accessible to a wide range of users.

Scenario 2: Health Care

A healthcare application that requires high-level security, reliability, and transaction support.

The best architecture pattern for this scenario is SOAP APIs. SOAP APIs are more secure than RESTful APIs, as they use a messaging protocol to ensure the integrity and security of data transmission. SOAP APIs also provide transaction support, which is essential for applications that require complex transactions, such as healthcare applications.

Scenario 3: Financial Services

A financial services application that requires high-level security and transaction support.
The best architecture pattern for this scenario is also SOAP APIs. SOAP APIs provide the necessary security and transaction support required by financial services applications. However, if the application also requires fast data retrieval, RESTful APIs can be used in conjunction with SOAP APIs to provide a comprehensive solution.

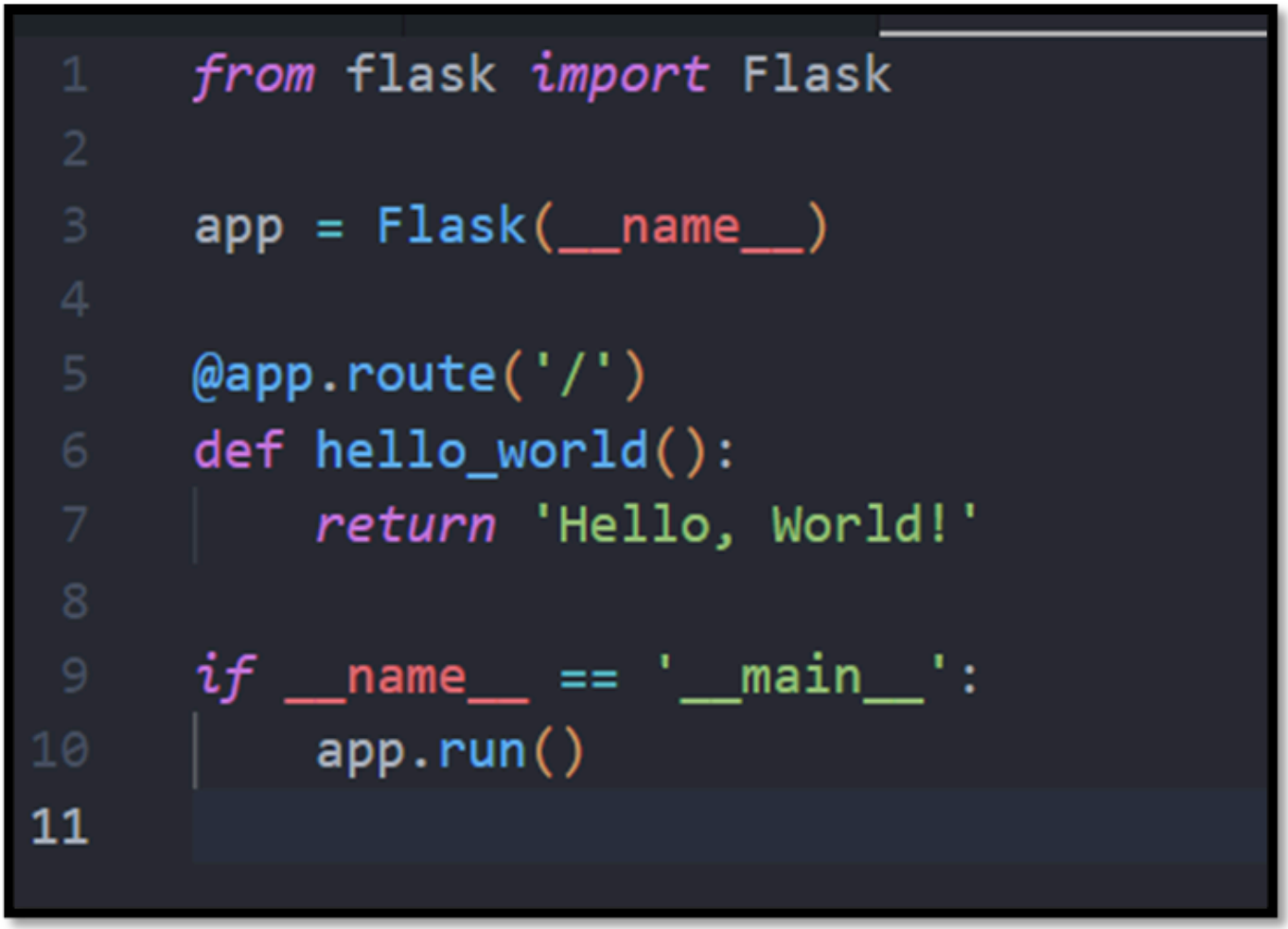
© Algonquin College 2022

Learning Activity 5.1: Sample Flask Application

With the downloads complete, let's create a basic Flask application, which will help us to better understand the fundamental concepts of REST API development using Flask. Following along with the following instructions.

Tasks:

- Create a new Python file, for example, app.py.
- Open it in a text editor or an integrated development environment (IDE).
- Add the following code to set up a simple Flask application:



```
1   from flask import Flask
2
3   app = Flask(__name__)
4
5   @app.route('/')
6   def hello_world():
7       return 'Hello, World!'
8
9   if __name__ == '__main__':
10      app.run()
11
```

- Save the file and run it using the following command in the command prompt or terminal:

python app.py

After having complete the learning activity, the Flask development server will start, and you can access the application by visiting **http://localhost:5000 (http://localhost:5000)** in your web browser. You should see the message "Hello, World!" displayed on the page.

Congratulations! You have successfully created a basic Flask application.

© Algonquin College 2022

Flask

Flask Python for REST API Development: A Comprehensive Guide

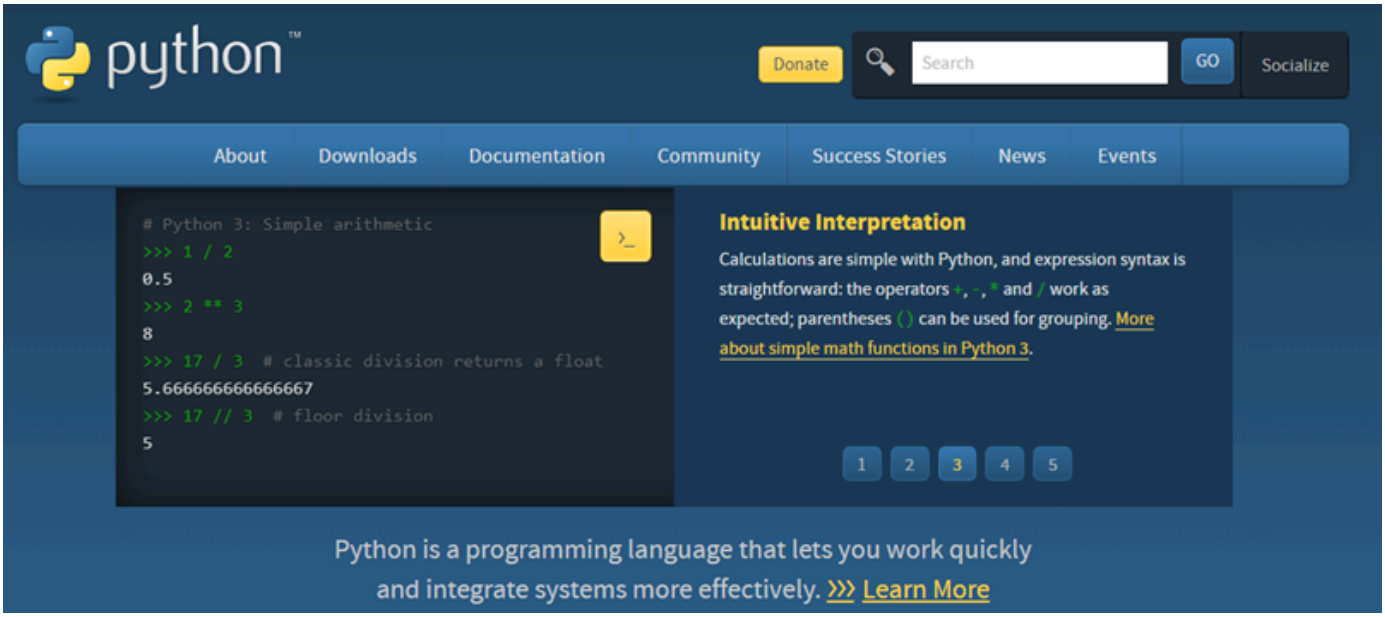
Flask is a popular Python web framework that allows developers to build web applications, including RESTful APIs, quickly and efficiently. In this module, we will explore the process of setting up Flask, creating a sample Flask application, and implementing routes and views for REST API development.

Installation of Python and Flask

Before we dive into Flask development, we need to ensure that Python and Flask are installed on our system. Follow the steps below to install Python and Flask:

1. Install Python

- Visit the official Python website at:
  - python.org.
- Navigate to the Downloads section and choose the latest Python version compatible with your operating system.



- Download the installer and run it.
- During the installation, make sure to check the box that says "Add Python to PATH" to ensure easy accessibility.
- Complete the installation by following the on-screen instructions.

The following video demonstrates how to install Python 3.10.0 on Windows 10. Please follow the instructions carefully: **https://www.youtube.com/watch?v=Kn1HF3oD19c (https://www.youtube.com/watch?v=Kn1HF3oD19c)**

2. Install Flask

Once Python is installed, we can proceed with installing Flask, using the following steps:

- Open a command prompt or terminal.
- Type the following command to install Flask using pip, which is a package manager for Python: ***pip install flask***
- Wait for the installation to complete. Flask and its dependencies will be installed automatically.

With Python and Flask successfully installed, we are ready to start building our Flask REST API application.

The following videos provide additional information about the Flask installation and its functionality, if needed.

- **https://www.youtube.com/watch?v=ojzNmvkNfqc (https://www.youtube.com/watch?v=ojzNmvkNfqc)**
- **https://www.youtube.com/watch?v=mqhxxeeTbu0 (https://www.youtube.com/watch?v=mqhxxeeTbu0)**

© Algonquin College 2022

Routes and Views

In Flask, routes are used to map URLs to functions, called views, which handle the requests and generate responses. Using the learning activity as your base, let's extend from our sample Flask application to include additional routes and views.

Modify the app.py file as follows:

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello_world():
    return 'Hello, World!'

@app.route('/api/users', methods=['GET'])
def get_users():
    # Logic to fetch and return a list of users
    return 'List of users'

@app.route('/api/users/<int:user_id>', methods=['GET'])
def get_user(user_id):
    # Logic to fetch and return a specific user based on user_id
    return f'Retrieving user with ID: {user_id}'

if __name__ == '__main__':
    app.run()
```

In this modified version, we have added two new routes: /api/users and /api/users/<int:user_id>. The first route responds to a GET request and returns a list of users.

The second route also responds to a GET request but includes a dynamic parameter user_id, allowing us to retrieve a specific user based on the provided ID.

Save the file and restart the Flask development server using the following command in the command prompt or terminal: ***python app.py***

Now, if you visit **http://localhost:5000/api/users (http://localhost:5000/api/users)** in your web browser, you should see the message "List of users" displayed. This indicates that the /api/users route is working correctly.

Similarly, you can access the /api/users/<user_id> route by providing a specific user_id in the URL. For example, visiting **http://localhost:5000/api/users/1 (http://localhost:5000/api/users/1)** will display the message "Retrieving user with ID: 1".

By adding routes and views to your Flask application, you can define the functionality and behavior of your REST API endpoints.

© Algonquin College 2022

Python Tutorial

This next part of the module might be review for some of you, but either way, it's beneficial in establishing foundational knowledge for what's to come.

The video below offers an excellent introduction to Python, covering its fundamental concepts and providing a comprehensive overview of the language.

- **https://www.youtube.com/watch?v=kqtD5dpn9C8 (https://www.youtube.com/watch?v=kqtD5dpn9C8)**

Before we get started working with Flask, it's essential to have a good understanding of the following topics in Python:

1. Python syntax and basic concepts: Familiarize yourself with the syntax, data types, variables, operators, control structures (if statements, loops), and functions in Python.

2. Object-oriented programming (OOP): Learn the principles of OOP in Python, including classes, objects, inheritance, and encapsulation. Flask utilizes OOP concepts for building web applications.

3. Modules and packages: Understand how to work with modules and packages in Python. Modules are files containing Python code, while packages are directories containing multiple modules. Flask itself is a package that you import and use in your Python code.

4. Working with data structures: Gain knowledge about lists, dictionaries, tuples, and sets, as they are commonly used in Flask applications for storing and manipulating data.

5. Understanding HTTP and RESTful principles: Familiarize yourself with the basics of the HTTP protocol and the principles of RESTful APIs. This knowledge will help you design and develop efficient and well-structured Flask APIs.

To learning more and/or familiarize yourself with these concepts, please ensure that you review the following resource as it discusses python fundamentals.

- **https://www.w3schools.com/python/ (https://www.w3schools.com/python/)**

© Algonquin College 2022

Python and Flask

We know that both Python and Flask are two popular technologies used for web development, but let's first examine them individually.

Python

Python is a high-level, interpreted programming language known for its simplicity and readability. It offers a wide range of libraries and frameworks that make it suitable for various applications, including web development, data analysis, artificial intelligence, and automation. Python emphasizes code readability, making it easier for developers to write and maintain their programs.

Flask

Flask is a micro web framework written in Python. It is designed to be simple and lightweight, providing developers with the essential tools to build web applications. Flask follows the Model-View-Controller (MVC) architectural pattern, allowing developers to separate their application's logic into different components. It provides routing, templating, and other essential features for web development, while still giving developers the flexibility to extend and customize their applications as needed.

When used together, Python and Flask provide a powerful combination for web development. Flask leverages the simplicity and flexibility of Python, making it an excellent choice for building web applications of various sizes and complexities.

Now, you might be thinking to yourself:

Why are we learning Flask?

Recall from the previous module that REST API is an ideal architecture for creating the API for IOT Devices. Because of that, we will be using the flask for creating the RESTAPI in the upcoming modules. Flask is an excellent choice for creating REST APIs. Flask provides the necessary tools and features to design and develop RESTful APIs efficiently.
Flask's simplicity, combined with its rich ecosystem of extensions, makes it a popular choice for creating REST APIs. It allows you to focus on implementing your application's business logic while providing the necessary tools for building robust and scalable APIs.

© Algonquin College 2022

In this module, we will learn about Python and Flask. Python and Flask are two popular technologies used for web development. Python is a versatile and beginner-friendly programming language, while Flask is a lightweight web framework written in Python.

Let's see what these technologies have to offer.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain Python and Flask technologies

02 Carryout the installation process of both Python and Flask

03 Exemplify the Flask application

04 Recognize the use of Routes and views

Key Terms and Concepts

1. API: Application Programming Interface
2. GraphQL: Graph Query Language
3. HTTP: Hypertext Transfer Protocol
4. IoT: Internet of Things
5. JSON: JavaScript Object Notation
6. REST: Representational State Transfer
7. SOAP: Simple Object Access Protocol
8. XML: eXtensible Markup Language

RESTful API Design

Today, Application Programming Interfaces (APIs) are at the heart of the digital revolution. Often described as currency in the digital world, APIs are especially valuable to retailers and brands for guaranteeing speed and safety in the exchange of information and delivering the agility needed to quickly build customer experiences. As we begin this module, the first thing we will examine are a few of the necessary elements of designing a RESTful API.

A. Identifying the IoT Device Resources and Functionalities

Before designing the API for IoT devices, it is crucial to identify the specific resources and functionalities that the API will interact with. This involves understanding the capabilities and characteristics of the IoT devices and determining the data and operations that can be performed on them. Some common IoT device resources and functionalities include:

- **Sensors:**

Identify the various sensors attached to the IoT devices, such as temperature sensors, humidity sensors, motion sensors, etc. Determine the types of data they provide and the frequency at which the data is updated.

- **Actuators:**

Determine the actuators present in the IoT devices, such as motors, switches, or LED lights. Define the actions that can be performed on these actuators, such as turning them on/off, controlling their speed, or adjusting their intensity.

- **Device Configuration:**

Identify the configuration settings that can be modified on the IoT devices, such as network settings, thresholds, sampling rates, or calibration values.

- **Device Management:**

Determine the functionalities related to device management, such as device registration, deregistration, firmware updates, or device status monitoring.

B. Defining the API Endpoints and URI Structure

Once the IoT device resources and functionalities have been identified, the next step is to define the API endpoints and URI structure. The URI (Uniform Resource Identifier) acts as a unique identifier for each resource exposed by the API. Here are some guidelines for defining the API endpoints and URI structure:

- **Resource-Oriented Approach:**

Adopt a resource-oriented approach where each IoT device resource is represented as a separate endpoint. For example, /devices, /sensors, /actuators.

- **Noun-Based Naming:**

Use nouns to name the endpoints that represent resources. For example, /devices/{device_id}, /sensors/{sensor_id}, /actuators/{actuator_id}.

- **Hierarchy and Nesting:**

If there are hierarchical relationships between resources, reflect them in the URI structure. For example, /devices/{device_id}/sensors/{sensor_id}.

- **CRUD Operations:**

Map the appropriate HTTP methods (GET, POST, PUT, DELETE) to the corresponding CRUD (Create, Read, Update, Delete) operations on the resources. For example, GET /devices to retrieve a list of devices, POST /devices to create a new device, PUT /devices/{device_id} to update a device, DELETE /devices/{device_id} to delete a device.

- **Action-Based Endpoints:**

In addition to resource-oriented endpoints, consider defining action-based endpoints for specific functionalities that do not fit into traditional CRUD operations. For example, /devices/{device_id}/activate, /devices/{device_id}/reset.

C. Determining the Appropriate HTTP Methods for Each Endpoint

The choice of HTTP methods for each endpoint depends on the desired operations to be performed on the resources. Here are some common HTTP methods used in IoT API design:

- **GET:**

Used to retrieve data from the server. Typically used for fetching sensor readings, device information, or specific configurations.

- **POST:**

Used to send data to the server for creating new resources. It is commonly used when registering new devices or creating new configurations.

- **PUT:**

Used to update existing resources or create resources with a specific identifier. It is often used for updating device configurations or modifying existing data.

- **DELETE:**

Used to remove resources from the server. It is employed when deleting devices, configurations, or specific data entries.

It is essential to choose the appropriate HTTP methods that align with the actions to be performed on the IoT device resources. Carefully consider the security implications and ensure that the chosen methods adhere to RESTful design principles.

By identifying IoT device resources, defining the API endpoints, and determining the appropriate HTTP methods, you can lay the foundation for a well-designed and functional API for IoT devices. This approach ensures that the API aligns with the specific functionalities and capabilities of the IoT devices, allowing for seamless interaction and control.

As you move forward with the API design, it's important to consider scalability, security, and ease of use. This involves carefully mapping the operations to the appropriate HTTP methods, ensuring that the URI structure is logical and intuitive, and considering any additional parameters or headers required for specific functionalities.

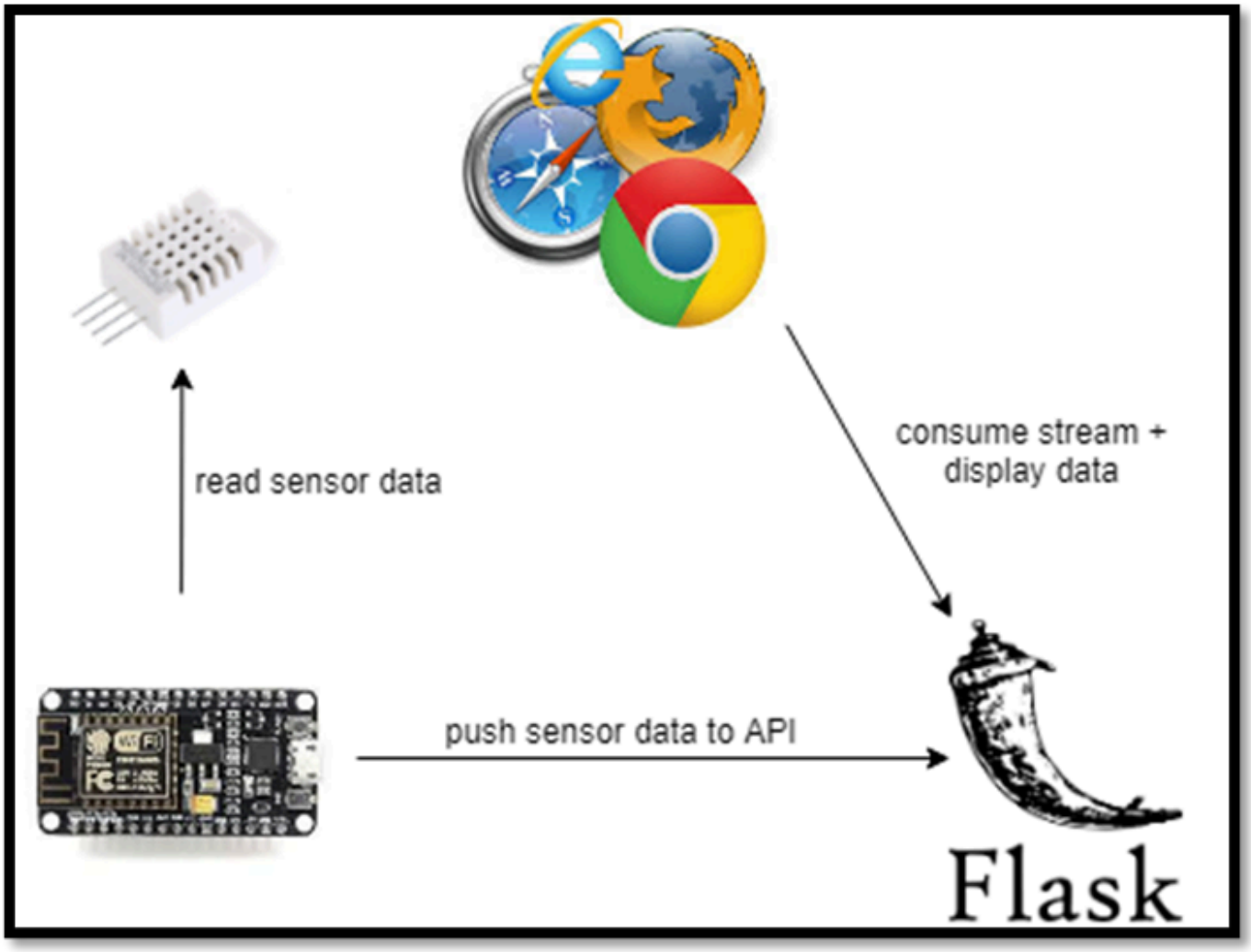
Furthermore, it's crucial to keep the API design flexible and extensible, as IoT devices and their functionalities may evolve over time. Plan for future enhancements and consider versioning strategies to ensure backward compatibility while introducing new features or improvements.

By following these guidelines and incorporating best practices for API design, you can create a robust and efficient API for IoT devices using the Flask Python framework. This API will serve as a powerful interface for interacting with and controlling IoT devices, enabling seamless integration into larger IoT ecosystems and applications.

© Algonquin College 2022

RESTful API Implementation

Start by taking a look at the following diagram.



Well, the diagram illustrates the connection between the weather sensor and the Raspberry Pi, which facilitates the transmission of data to Flask. Subsequently, the data can be accessed and viewed using web browsers. It's important to note that this course primarily focuses on the software aspect rather than the integration of IoT hardware.
However, participants are encouraged to explore hardware integration with their REST API if they desire. As we are not directly connected to a physical sensor, we will simulate the functionality by generating sample data for the weather sensor readings.

Let's create sample data for an IoT weather-based sensor and then develop a REST API using Flask to handle the data. In this example, we'll consider temperature, humidity, and pressure readings from the sensor.

Example



```
[
    {
      "timestamp": "2023-05-28T10:30:00",
      "temperature": 25.5,
      "humidity": 60.2,
      "pressure": 1012.3
    },
    {
      "timestamp": "2023-05-28T10:35:00",
      "temperature": 25.8,
      "humidity": 59.8,
      "pressure": 1012.7
    },
    {
      "timestamp": "2023-05-28T10:40:00",
      "temperature": 26.1,
      "humidity": 59.5,
      "pressure": 1012.9
    }
  ]
```

In this example, we have a list of sensor data entries. Each entry contains a timestamp (in ISO 8601 format), temperature (in degrees Celsius), humidity (in percentage), and pressure (in hPa).

Now, let's create a Flask application and implement the REST API to handle the data:

```python
from flask import Flask, jsonify
app = Flask(__name__)

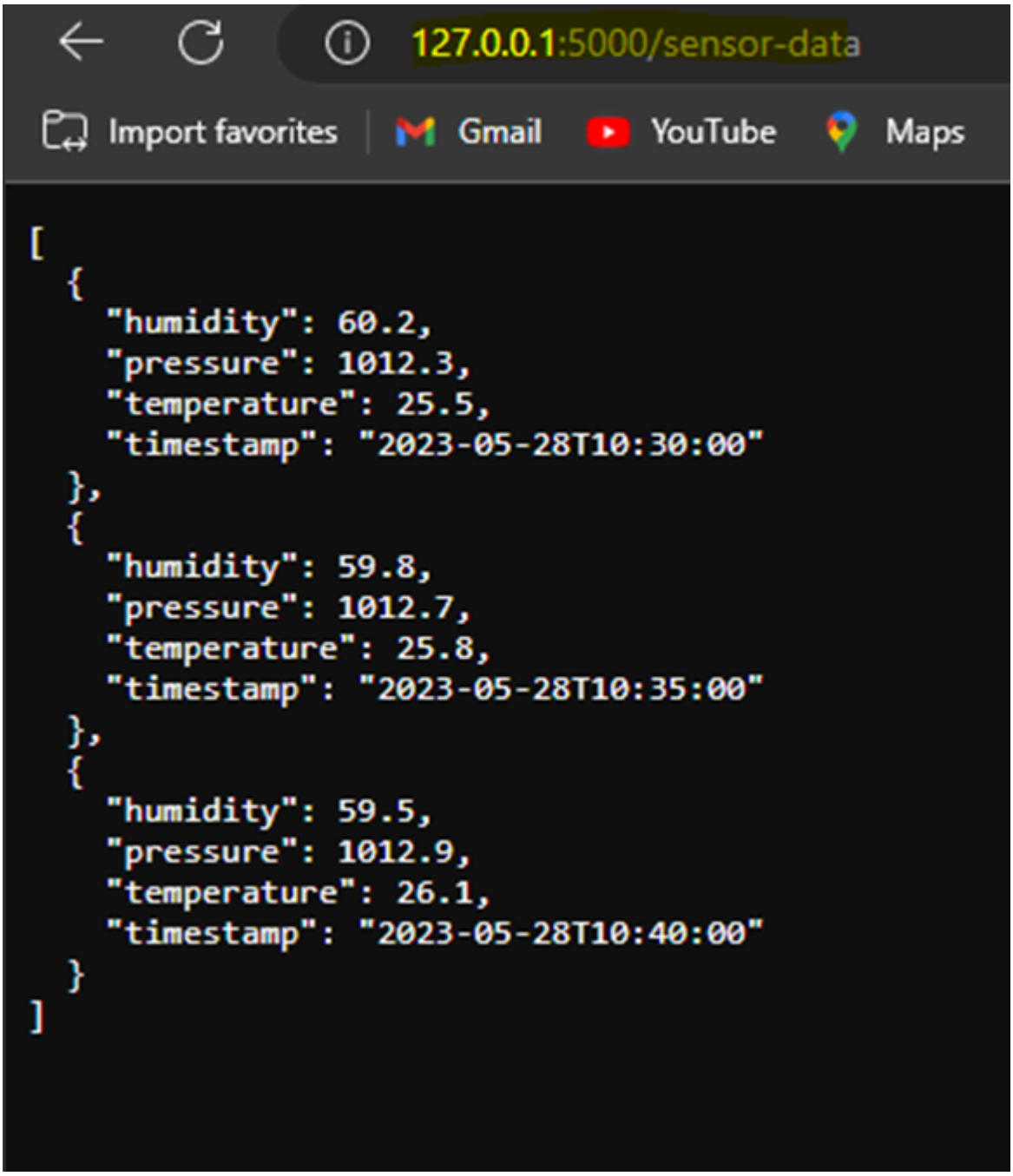
sensor_data = [
    {
        "timestamp": "2023-05-28T10:30:00",
        "temperature": 25.5,
        "humidity": 60.2,
        "pressure": 1012.3
    },
    {
        "timestamp": "2023-05-28T10:35:00",
        "temperature": 25.8,
        "humidity": 59.8,
        "pressure": 1012.7
    },
    {
        "timestamp": "2023-05-28T10:40:00",
        "temperature": 26.1,
        "humidity": 59.5,
        "pressure": 1012.9
    }
]

@app.route('/sensor-data', methods=['GET'])
def get_sensor_data():
    return jsonify(sensor_data)

if __name__ == '__main__':
    app.run(debug=True)
```

Here:

- We import the necessary modules and create a Flask application. We define a list called sensor_data that contains the sample sensor data entries.
- Next, we define a route /sensor-data and associate it with the get_sensor_data function using the @app.route decorator. This function returns the sensor_data list in JSON format using the jsonify function from Flask.
- Finally, we run the Flask application using app.run().
- When you run this application and access the URL **http://localhost:5000/sensor-data (http://localhost:5000/sensor-data)** in your browser or send a GET request to that URL, you will receive the sensor data in JSON format.



Note: Make sure to install jsonify (pip install jsonify) before running the application.

To review some of the fundamental principles and guidelines for implementing a RESTful API, including identifying resources, defining endpoints, choosing HTTP methods, and implementing with Flask, please watch the following video: **https://www.youtube.com/watch?v=GMppyAPbLYk&t=1712s (https://www.youtube.com/watch?v=GMppyAPbLYk&t=1712s)**

© Algonquin College 2022

In this module, we will explore the process of developing a RESTful API specifically designed for IoT devices. We will leverage the Flask Python framework to build a robust and scalable API that can interact with IoT devices, retrieve sensor data, control actuators, and manage device resources. By the end of this module, you will have a solid understanding of the fundamentals of developing REST APIs for IoT and be able to apply that knowledge to your own IoT projects.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain RESTful API design

02 Design a RESTful API

03 Implement a RESTful API

04 Deconstruct best practices for RESTful API design

Key Terms and Concepts

1. API: Application Programming Interface
2. IoT: Internet of Things
3. REST: Representational State Transfer
4. SOAP: Simple Object Access Protocol
5. GraphQL: Graph Query Language
6. XML: eXtensible Markup Language
7. JSON: JavaScript Object Notation
8. HTTP: Hypertext Transfer Protocol



Distributed Application State

In modern IoT systems, the ability to support real-time changes to the state of connected devices is critical. Users expect IoT applications to respond immediately to their commands, update device status across multiple platforms, and provide a seamless and interactive experience. To achieve this, developers employ various approaches and technologies that enable real-time communication and synchronization of device state. In this module, we will explore these techniques and their implementation details in the context of IoT applications.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain real-time changes to distributed application states

02 Outline Cloud infrastructure components for supporting real-time changes.

03 Recognize different types of APIs such as WebSockets and Server-Sent Events

Key Terms and Concepts

- API: Application Programming Interface
- GraphQL: Graph Query Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- REST: Representational State Transfer
- SOAP: Simple Object Access Protocol
- XML: eXtensible Markup Language



WebSocket Vs MQTT Vs HTTPS

When choosing between WebSocket, MQTT, and HTTPS for IoT devices, one must consider the specific requirements and constraints of your IoT application. Let's consider the use cases and characteristics of each protocol to help you make an informed decision:

1. **WebSocket:**

WebSocket is a full-duplex communication protocol that provides real-time, bidirectional communication between clients and servers. It is well-suited for scenarios where you require low-latency, real-time updates and direct interaction between devices and servers. WebSocket can be a good choice for IoT applications that involve real-time control or continuous data streaming.

Real-time Control and Monitoring:

In a smart home automation system, WebSocket can be used to enable real-time control and monitoring of devices. For instance, a user can use a mobile application to control lights, temperature, and security systems in their home. WebSocket allows instant updates and bidirectional communication between the mobile app and the smart home hub, providing a seamless user experience.

Advantages:

- Low-latency and real-time communication.
- Full-duplex bi-directional communication.
- Efficient for scenarios that require frequent updates.
- Eliminates the need for constant polling.

Considerations:

- Requires a WebSocket server implementation.
- May consume more resources (memory, processing power) compared to MQTT in resource-constrained devices.
- WebSocket connections may not be available or supported in some network environments.

2. **MQTT:**

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol designed for efficient communication in low-bandwidth, high-latency networks. It follows a publish/subscribe model, where devices publish messages to topics, and other devices subscribe to those topics to receive updates. MQTT is widely used in IoT applications due to its small footprint and efficient data transmission.

Smart Energy Management:

In a smart energy grid, MQTT can be used to facilitate communication between smart meters, energy monitoring devices, and the central grid management system. Smart meters publish energy consumption data to a specific MQTT topic, and the grid management system subscribes to these topics to receive real-time updates on energy usage. MQTT's lightweight nature makes it ideal for low-power devices and networks with limited bandwidth.

Advantages:

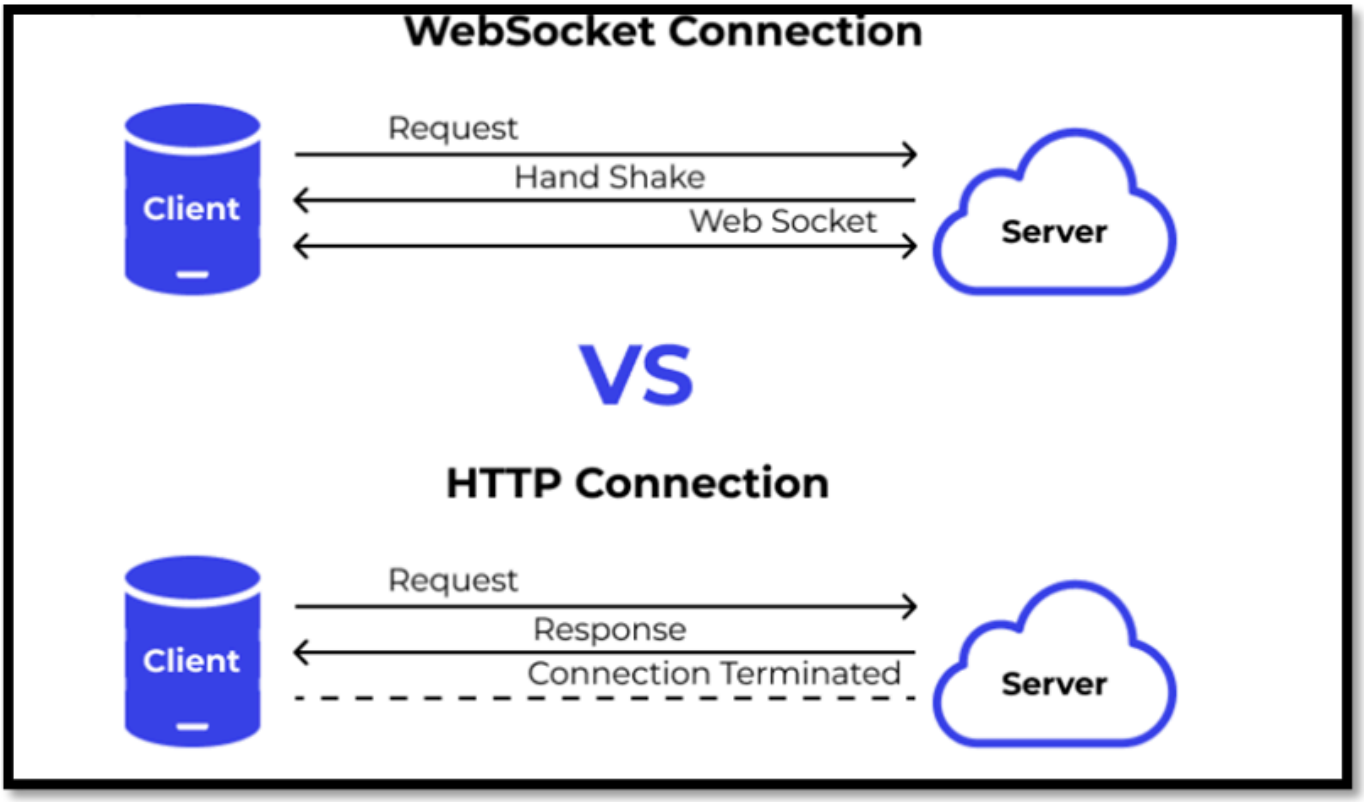
- Lightweight and efficient protocol, suitable for resource-constrained devices.
- Supports a publish/subscribe model for scalable communication.
- Reliable message delivery with configurable Quality of Service (QoS) levels.
- Well-suited for scenarios with intermittent connectivity and limited bandwidth.

Considerations:

- Requires an MQTT broker for message routing and delivery.
- Not as well-suited for scenarios requiring real-time, bidirectional communication compared to WebSocket.
- May require additional security measures (e.g., encryption) depending on the MQTT implementation.

3. HTTPS:

HTTPS (Hypertext Transfer Protocol Secure) is the secure version of HTTP and is commonly used for web-based communication. While not specifically designed for IoT applications, HTTPS can be used for secure communication between IoT devices and servers, especially when data confidentiality and integrity are crucial.



Secure IoT Data Transmission:

In an industrial IoT setting, HTTPS can be utilized to ensure secure data transmission between sensors, actuators, and a central server. For example, in a smart manufacturing environment, temperature and humidity sensors can securely transmit data to a cloud-based analytics platform over HTTPS. The encryption provided by HTTPS helps protect sensitive data from unauthorized access or tampering.

Advantages:

- Wide adoption and support in web environments.
- Standardized security measures with SSL/TLS encryption.
- Compatible with existing web infrastructure and security practices.
- Suitable for scenarios where secure communication is a priority.

Considerations:

- May have higher network overhead compared to WebSocket or MQTT due to HTTP headers.
- May not be as efficient for resource-constrained devices.
- Not specifically designed for real-time or publish/subscribe communication patterns.

In summary, if your IoT application requires real-time, bidirectional communication and frequent updates, WebSocket may be a good choice. If you have resource-constrained devices and prioritize lightweight, efficient messaging with a publish/subscribe model, MQTT is a suitable option. HTTPS

is more focused on security and compatibility with existing web infrastructure, making it suitable when secure communication is crucial.

It's important to note that IoT applications often require a combination of these protocols. For instance, WebSocket may be used for real-time control, while MQTT is employed for data telemetry and remote monitoring. HTTPS can be utilized to secure communication channels and provide authentication in both WebSocket and MQTT implementations. The choice of protocols depends on the specific requirements, communication patterns, and security considerations of the IoT application.

© Algonquin College 2022

Real-Time Communication Protocols

One of the key technologies for supporting real-time changes in distributed applications is the use of real-time communication protocols. These protocols allow for bi-directional, low-latency communication between clients and servers, enabling real-time data transmission and updates.

Real-Time Communication Protocols include:

a. Websockets
b. Server-Sent Events (SSE)
c. MQTT (Message Queuing Telemetry Transport)

Let's take a look at each of these protocols in greater detail.

- **WebSocket**

WebSocket is a communication protocol that provides full-duplex, bidirectional communication channels over a single TCP connection. Unlike traditional request-response protocols, WebSocket allows for real-time, low-latency communication between clients and servers. It establishes a persistent connection, enabling both the client and server to send data to each other at any time. To learn more about WebSockets, please watch the following video.

WebSocket is particularly suitable for real-time applications that require frequent updates, such as chat applications, collaborative editing tools, and real-time dashboards. It eliminates the need for constant polling or long-polling techniques, reducing network overhead and improving responsiveness.

- **https://www.youtube.com/watch?v=1BfCnjr_Vjg (https://www.youtube.com/watch?v=1BfCnjr_Vjg)**

- **Server-Sent Events (SSE)**

Server-Sent Events (SSE) is another real-time communication protocol that enables servers to push data to clients over HTTP. SSE allows the server to send a stream of events as a response to a client request, establishing a persistent connection. The server can then push updates to the client whenever new events occur.

SSE is well-suited for scenarios where the server primarily pushes updates to clients, such as real-time notification systems, live streaming, and stock market tickers. It provides a lightweight and straightforward mechanism for delivering real-time updates without the need for complex client-server interactions. For additional information on SSE, please watch the following video: **https://www.youtube.com/watch?v=Z4ni7Gsilbs (https://www.youtube.com/watch?v=Z4ni7Gsilbs)**

- **MQTT (Message Queuing Telemetry Transport)**

MQTT is a lightweight messaging protocol designed for efficient communication between devices in low-bandwidth, high-latency networks. It follows a publish/subscribe model, where publishers send messages to topics, and subscribers receive messages from those topics.

MQTT is commonly used in Internet of Things (IoT) applications, where devices with limited resources need to exchange real-time data. It has a small footprint, minimal overhead, and supports Quality of Service (QoS) levels for reliable message delivery. MQTT brokers act as intermediaries, facilitating the exchange of messages between publishers and subscribers. For more on MQTT, please watch the following video: **https://www.youtube.com/watch?v=6XqGXw_5NhM (https://www.youtube.com/watch?v=6XqGXw_5NhM)**

© Algonquin College 2022

WebSocket API Design in Flask

The WebSocket API design involves defining the endpoints and the corresponding actions that can be performed through the WebSocket connection. Here are some key considerations for designing a WebSocket API:

1.1 Endpoints:

- Determine the endpoints that will be available for WebSocket communication. Endpoints represent different components or functionalities of your application.
- Choose descriptive and intuitive endpoint names that reflect the purpose of each endpoint.
- Assign a unique URL or path to each endpoint to differentiate them.

1.2 Actions

- Define the actions or operations that can be performed on each endpoint. Actions represent the specific tasks or commands that clients can execute.
- Identify the various actions that make sense for each endpoint, based on the functionalities of the corresponding component.
- Use action names that are clear and concise, reflecting the intended operation.

1.3 Message Format:

- Determine the structure and format of the messages exchanged between the server and clients over the WebSocket connection.
- Specify the data fields that will be included in each message, such as the endpoint, action, and payload.
- Consider using a standardized data format like JSON to ensure consistency and interoperability.

1.4 Payload

- Define the payload format for messages that require additional data or parameters.
- Determine the specific fields or properties needed to carry out each action effectively.
- In the case of the /ws/lights endpoint, the payload could include information such as the desired state of the lights (e.g., on or off), brightness level, color settings, or any other relevant parameters needed to control the lights.

- Consider using a schema or documentation to provide clear guidelines for constructing the payload.

1.5 Event Handling:

- Establish the event handling mechanism for the WebSocket API.
- Define how the server will respond to different events, such as connection establishment, disconnection, or incoming messages.
- Implement appropriate event handlers on the server side to process and respond to client requests or notifications.

1.6 Error Handling:

- Define a consistent approach for handling errors and communicating error conditions to clients.
- Specify error codes or status indicators that can be included in error responses to provide meaningful feedback to clients.
- Consider providing descriptive error messages or explanations to aid in troubleshooting and debugging.

By carefully designing the WebSocket API, you can provide a clear and efficient communication interface that allows clients to interact with your server-side application in real-time.

Scenario continued…

Using the same smart home automation scenario as earlier, let's see how WebSocket API design using Flask is implemented. In the smart home automation scenario, we can define the following WebSocket API endpoints:

- /ws: This endpoint handles the WebSocket connection and serves as the entry point for communication between the server and the IoT devices.
- /ws/lights: This endpoint handles commands related to the lights in the smart home
- /ws/thermostat: This endpoint handles commands related to the thermostat.
- /ws/cameras: This endpoint handles commands related to the security cameras.

Each endpoint represents a specific IoT device or functionality within the smart home system.

Implementing the WebSocket API in Flask:

To implement the WebSocket API in Flask, you can use the Flask-SocketIO extension, which provides WebSocket support for Flask applications.

- The Flask-SocketIO extension is a Flask extension that adds WebSocket support to Flask applications. It allows you to easily integrate WebSocket communication into your Flask-based web application.

- To use Flask-SocketIO, you need to install it first. You can install it via pip using the following command: pip install flask-socketio

- Once installed, you can start using Flask-SocketIO in your Flask application. Here's a small example to demonstrate how to use Flask-SocketIO:

app.py > ...

```python
from flask import Flask, render_template
from flask_socketio import SocketIO, emit

app = Flask(__name__)
# Set a secret key for the application
app.config['SECRET_KEY'] = 'your_secret_key'
socketio = SocketIO(app)

@app.route('/')
def index():
    return render_template('index.html')  # Render the HTML template

@socketio.on('connect')
def handle_connect():
    print('New WebSocket connection established')

@socketio.on('disconnect')
def handle_disconnect():
    print('WebSocket connection closed')

@socketio.on('message')
def handle_message(data):
    print('Received message:', data)
    # Emit a WebSocket event to clients
    emit('response', 'Server received your message')

if __name__ == '__main__':
    socketio.run(app, debug=True, allow_unsafe_werkzeug=True)
```

- from flask import Flask, render_template: Importing the necessary modules

- app = Flask(__name__): Creating an instance of the Flask application

- app.config['SECRET_KEY'] = 'your_secret_key': Setting a secret key for the application. This is necessary for some Flask extensions, including Flask-SocketIO.

- socketio = SocketIO(app): Creating an instance of the SocketIO extension, passing the Flask application as the argument.
- @app.route('/'): Defining a route handler for the root URL ("/"). This is a decorator that associates the following function with the specified route.
- def index(): The function that handles the root URL. It returns the rendered index.html template. This is the default page that will be displayed when accessing the root URL.
- @socketio.on('connect'): A decorator that associates the following function with the 'connect' event. It is triggered when a new WebSocket connection is established.
- def handle_connect(): The function that handles the 'connect' event. It prints a message to the console indicating a new WebSocket connection has been established.
- @socketio.on('disconnect'): A decorator that associates the following function with the 'disconnect' event. It is triggered when a WebSocket connection is closed.
- def handle_disconnect(): The function that handles the 'disconnect' event. It prints a message to the console indicating that the WebSocket connection has been closed.
- @socketio.on('message'): A decorator that associates the following function with the 'message' event. It is triggered when a client sends a message to the server.
- def handle_message(data): The function that handles the 'message' event. It prints the received message to the console and emits a 'response' event back to the client using emit(). The 'response' event carries the message 'Server received your message'.
- if __name__ == '__main__': A conditional block that ensures the Flask-SocketIO application is only run if the script is executed directly (not imported as a module).
- socketio.run(app, debug=True, allow_unsafe_werkzeug=True): Starts the Flask-SocketIO application, allowing it to accept WebSocket connections. The debug=True parameter enables debug mode, and allow_unsafe_werkzeug=True allows the use of the Werkzeug development server even in debug mode.

Here's an example of how to implement the WebSocket using Flask and Flask-SocketIO: **https://www.youtube.com/watch?v=AMp6hlA8xKA&t=408s (https://www.youtube.com/watch?v=AMp6hlA8xKA&t=408s)**

© Algonquin College 2022

WebSocket

WebSockets are a communication protocol that provides full-duplex, bidirectional communication channels over a single TCP connection. Unlike traditional HTTP, which follows a request-response model, WebSockets enable continuous, real-time communication between clients and servers.

In the context of IoT (Internet of Things) devices, WebSockets play a crucial role in facilitating efficient and interactive communication. Here's how WebSockets are used in IoT and a real-life scenario:

1. **Efficient and Real-Time Communication:**

WebSockets offer a persistent connection between IoT devices and the server, allowing them to exchange data in real time. Unlike HTTP, which requires establishing a new connection for each request, WebSockets maintain an open connection, eliminating the overhead of establishing connections repeatedly. This efficient and persistent connection enables immediate data updates and reduces latency, making it ideal for IoT applications.

2. **Bi-Directional Communication:**

WebSockets support bi-directional communication, meaning both the IoT device and the server can send and receive data simultaneously. This capability allows IoT devices to send sensor data to the server and receive commands or updates from the server in real time. Bi-directional communication is crucial for scenarios where devices need to react quickly to commands or receive timely updates from the server.

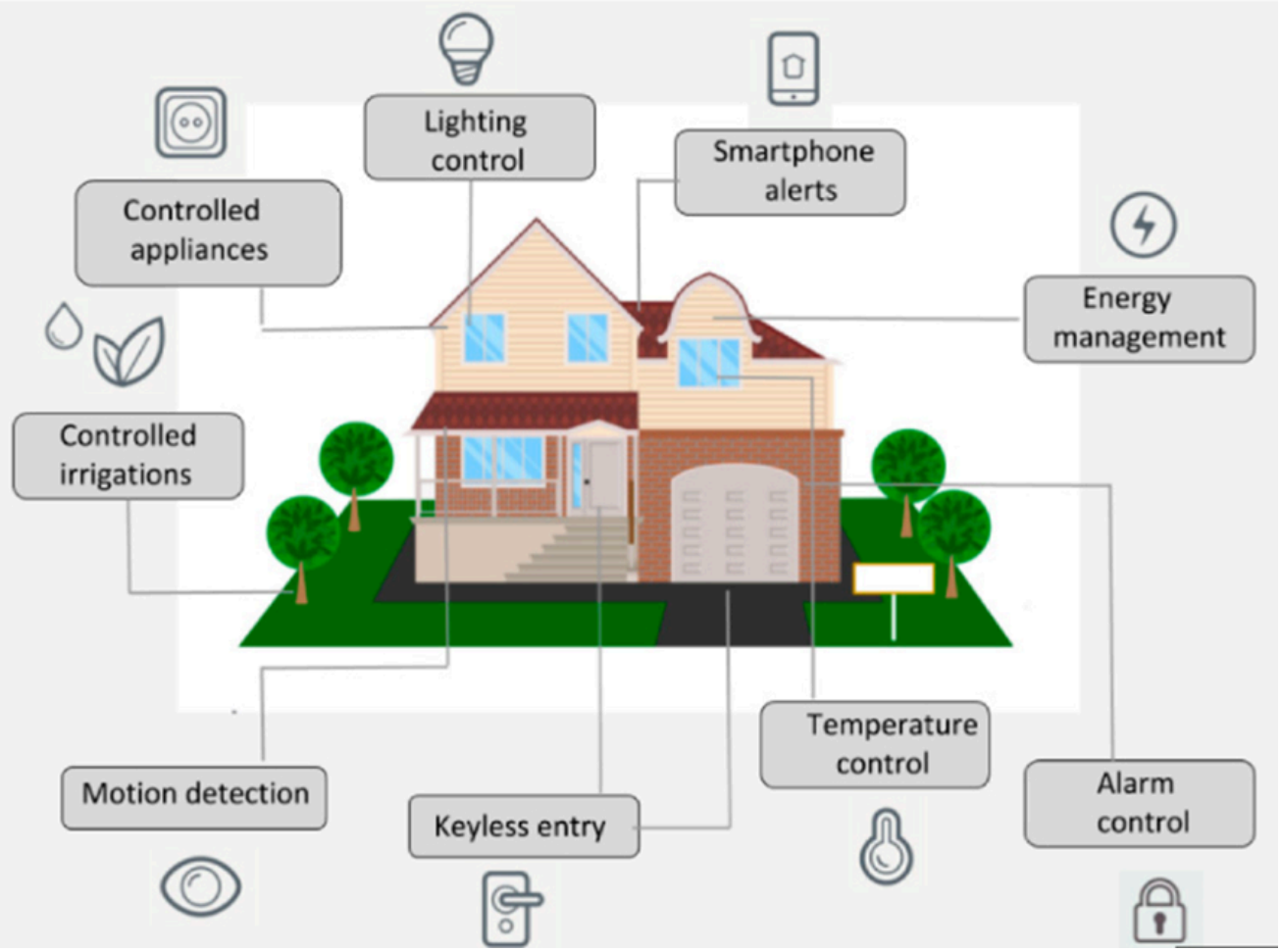
3. **Event-Based Communication:**

WebSockets use an event-driven model, where the server can push data to connected devices without the need for them to send explicit requests. This event-based communication allows IoT devices to subscribe to specific topics or channels and receive relevant data whenever it becomes available. For example, a weather station IoT device can subscribe to a channel that provides real-time weather updates, receiving data whenever there is a new forecast or weather alert.

Read the following real life situation to help clarify.

Scenario: Smart Home Automation

Imagine a smart home automation system where various IoT devices, such as lights, thermostats, and security cameras, are connected to a central server. WebSockets can be employed to enable real-time communication between the devices and the server, allowing for seamless control and monitoring of the smart home.



- Using WebSockets, the server can send commands to the IoT devices to turn the lights on or off, adjust the temperature, or activate/deactivate security cameras. The devices can instantly respond to these commands, providing real-time feedback to the server. For example, when a user triggers a command to turn off the lights, the server sends the command through WebSockets, and the lights receive the command, instantly turning off and sending a confirmation back to the server.

- Additionally, WebSockets can be used to stream data from IoT devices to the server, such as sensor readings, energy consumption, or security alerts. The server can monitor this data in real time, make decisions based on the incoming information, and send appropriate commands back to the devices as necessary. For instance, if a security camera detects movement, it can send a notification to the server using WebSockets, which can then trigger an alarm or notify the homeowner through a mobile app.

- In this smart home scenario, WebSockets enable fast, bidirectional, and event-based communication between the IoT devices and the server, resulting in a seamless and interactive user experience.

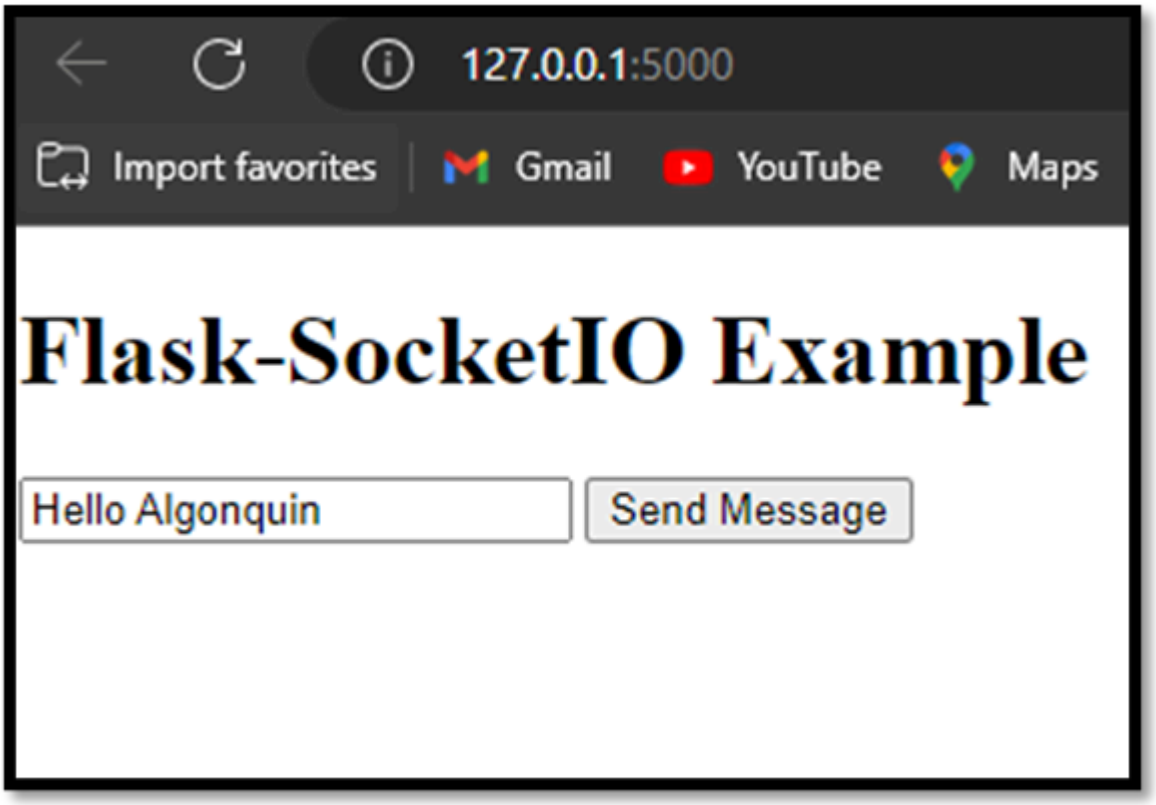
© Algonquin College 2022

Learning Activity 6.1: WebSocket using Flask

Now, it's your turn to give it a try. To get the above program on your computer, complete the following steps:

1. Download the Module 6 code.zip here (Module%206%20code.zip?isCourseFile=true&ou=890980)

2. Index.html file is placed inside a folder named templates in the same directory as the app.py file. The directory structure should look like this:



3. Open a terminal or command prompt and navigate to the directory where the app.py file is located.

4. Ensure that you have the necessary dependencies installed.

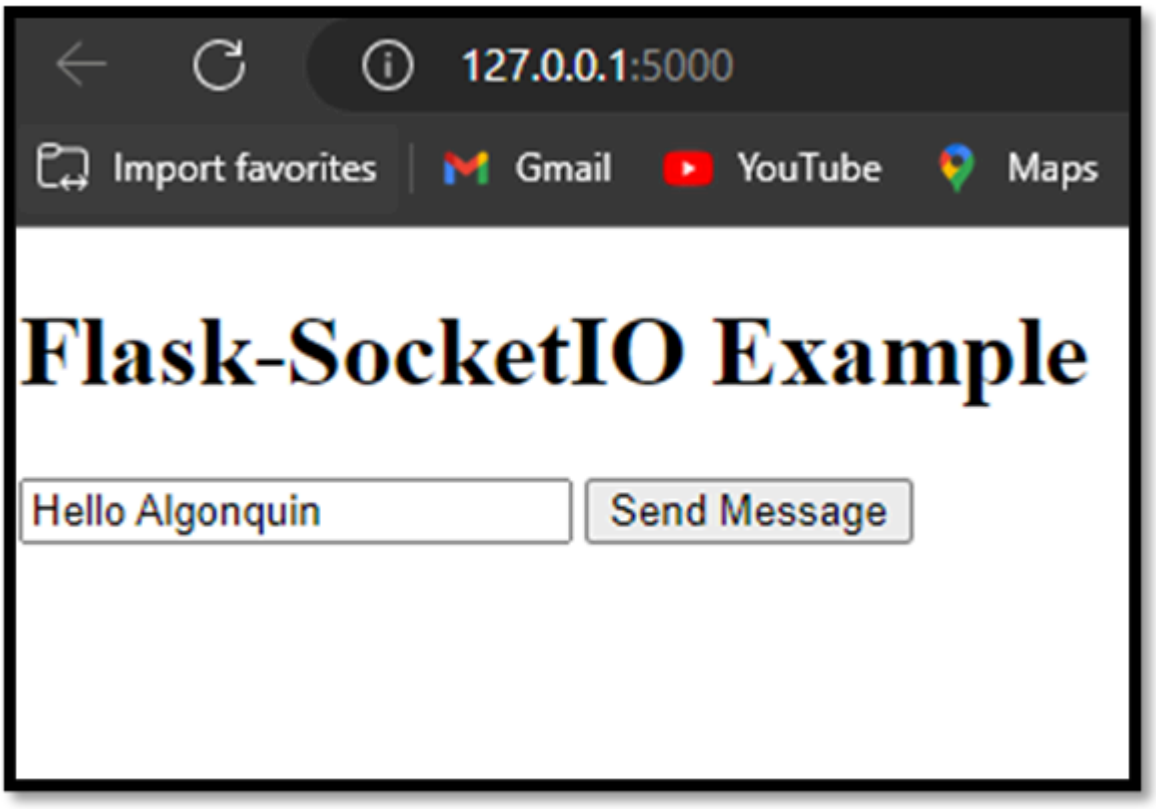
5. Once the dependencies are installed, you can start the Flask application

6. The Flask application will start running, and you should see output indicating that the server is running. It will typically be **http://127.0.0.1:5000/ (http://127.0.0.1:5000/)** or **http://localhost:5000/ (http://localhost:5000/)**.

7. Open a web browser and navigate to the URL provided in the console output (e.g., **http://127.0.0.1:5000/) (http://127.0.0.1:5000/))**.

8. You should see the rendered index.html template in the browser, indicating that the Flask application is working correctly.

9. Familiarize yourself with the program.



© Algonquin College 2022

WebSocket is a powerful communication protocol that enables real-time, bidirectional communication between clients and servers. In this module, we will explore the design and implementation of WebSocket APIs for supporting real-time changes to distributed application state.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain WebSocket

02 Design WebSocket APIs for enabling real-time communication.

03 Implement WebSocket API endpoints to handle data transmission and updates.

Key Terms and Concepts

- API: Application Programming Interface
- GraphQL: Graph Query Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- REST: Representational State Transfer
- SOAP: Simple Object Access Protocol
- XML: eXtensible Markup Language

Code Example: Retry mechanism

Here is an example of a Flask REST API with a retry mechanism for handling intermittent network availability in an IoT device.

Example

- Here, we have a Flask application with a single endpoint (/) that returns a JSON response indicating whether the IoT device is connected to the network or not.
- The connect_to_network() function simulates the connection process. It tries to connect to the network by checking the network_available flag and retrying until either the network becomes available or the maximum number of retries is reached. The number of retries and the time interval between retries can be adjusted according to your requirements.
- Upon running the application (python app.py), the connect_to_network() function is called before starting the Flask server. It will attempt to connect to the network and print the connection status.
- You can test the REST API by sending a GET request to **http://localhost:5000/ (http://localhost:5000/)**. The response will indicate whether the IoT device is connected to the network or not.
- This simple example demonstrates how you can incorporate a retry mechanism in a Flask REST API to handle intermittent network availability in an IoT device.
  Using the above steps and the provided zip file along with the code below, you can now see how to handle intermittent network availability.

Using the above steps and the provided zip file along with the code below, you can now see how to handle intermittent network availability.

- **Download .zip file here (Module%207%20code.zip?isCourseFile=true&ou=890980)**



```python
import time
from flask import Flask, jsonify

app = Flask(__name__)
# Global variable to simulate network availability
network_available = True
# Maximum number of retries
max_retries = 5

@app.route('/')
def index():
    return jsonify({'message': "IoT device is connected to the network" if network_available else
                    "IoT device is not connected to the network"})


def connect_to_network():
    retries = 0
    while not network_available and retries < max_retries:
        print(
            f"Attempting to connect to the network... (Attempt {retries + 1}/{max_retries})")
        time.sleep(1)  # Wait for 1 second before retrying
        retries += 1

    if network_available:
        print("Connected to the network!")
    else:
        print("Failed to connect to the network.")


if __name__ == '__main__':
    connect_to_network()
    app.run(debug=True)
```

© Algonquin College 2022

Intermittent network availability

Outlined here is the approach for handling intermittent network availability in IoT devices:

1. **Connection Monitoring:**

Implement a mechanism in the IoT device to continuously monitor the status of the network connection. This can be done by periodically sending heartbeat messages to a central server or gateway and expecting a response within a specified time frame. If a response is not received, it indicates a network connection issue.

2. **Retry Mechanism:**

When a network connection issue is detected, the IoT device should initiate a retry mechanism to establish a new connection. The device can attempt to reconnect at regular intervals, applying backoff strategies to increase the delay between reconnection attempts. This allows the device to adapt to varying network conditions and improve the chances of successfully reconnecting.

3. **Local Data Storage:**

Equip the IoT device with local storage capabilities to store data when a network disruption occurs. The device can cache the data locally until the connection is restored. Once the connection is established, the device can synchronize the stored data with the central server or gateway.

4. **Asynchronous Processing:**

Implement asynchronous programming techniques in the IoT device to handle multiple tasks concurrently. This allows the device to continue processing other tasks or sensor data while waiting for the network connection to be reestablished. Asynchronous processing helps maximize device efficiency and responsiveness even during intermittent network availability.

5. **Error Handling and Logging:**

Implement robust error handling and logging mechanisms in the IoT device. This includes logging network-related errors, connection failures, and recovery attempts. Error logs can be used for troubleshooting and identifying patterns of network disruptions. Additionally, error notifications can be sent to administrators or users to alert them about the network connectivity issues.

6. **Graceful Degradation:**

Design the IoT system to gracefully degrade its functionality in the event of intermittent network availability. For example, if real-time data transmission is not possible, the device can switch to a local mode where it operates with cached data or predefined rules until the network connection is restored. Graceful degradation ensures that the device continues to provide essential functionality and avoids abrupt disruptions in its operation.

By incorporating these strategies into the design and implementation of IoT devices, intermittent network availability can be effectively handled. These approaches help ensure that data is captured and synchronized when network connections are available, devices can adapt to varying network conditions, and system functionality is maintained even during temporary disruptions.

© Algonquin College 2022

IoT

In IoT applications, intermittent network availability refers to situations where the connection between IoT devices and the server is not consistently stable. This can happen due to various factors such as signal interference, network congestion, or device mobility. Intermittent network availability can pose challenges in maintaining reliable and continuous communication between IoT devices and the server. However, with proper design and handling techniques, these challenges can be mitigated.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain intermittent network availability

02 Recognize the key considerations for planning end-to-end application architecture.

03 Explain how to reach a seamless user experience and avoid data loss

Key Terms and Concepts

- API: Application Programming Interface
- GraphQL: Graph Query Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- REST: Representational State Transfer
- SOAP: Simple Object Access Protocol
- XML: eXtensible Markup Language



MQTT Vs HTTP Vs WebSockets

With that introduction to MQTT, let's compare it to HTTP and WebSockets.

MQTT

- Publish/subscribe model:

MQTT follows a publish/subscribe messaging pattern. Clients (publishers) send messages to a central broker, which then distributes the messages to interested subscribers based on the topic they have subscribed to.

- Lightweight and efficient:

MQTT is designed to be lightweight and efficient, making it suitable for devices with limited processing power and network bandwidth. It uses a binary protocol over TCP/IP, reducing the overhead of message headers and making it more efficient than protocols like HTTP.

- Asynchronous communication:

MQTT operates on an asynchronous model, where clients can send and receive messages independently without waiting for a response. This makes it well-suited for scenarios where real-time or near real-time data exchange is required.

- QoS levels:

MQTT provides different levels of Quality of Service (QoS) for message delivery. It allows clients to choose the desired level of reliability, ranging from "at most once" (QoS 0) to "at least once" (QoS 1) to "exactly once" (QoS 2). Let's have a look at each of these levels of reliability.

1. QoS 0 (At most once): Messages are delivered to the broker once without acknowledgment or guarantee of delivery. Lightweight and suitable for non-critical or easily replaceable data.

2. QoS 1 (At least once): Messages are guaranteed to be delivered at least once to the subscribers. The broker acknowledges receipt and retransmits if necessary. Suitable for scenarios where message reliability is important, despite the possibility of duplicate messages.

3. QoS 2 (Exactly once): Messages are guaranteed to be delivered exactly once to the subscribers. Involves a four-step handshake process to avoid message loss and duplicates. Most reliable but introduces additional overhead and complexity.

QoS 0 is the least reliable but most lightweight option. QoS 1 ensures at least once delivery with the possibility of duplicates. QoS 2 provides the highest level of reliability by guaranteeing exactly once delivery, albeit with increased overhead and complexity. The choice of QoS level depends on the specific needs and requirements of your application.

HTTP (Hypertext Transfer Protocol)

- Request/response model:

HTTP follows a request/response model, where a client sends a request to a server, and the server responds with the requested data. It is primarily used for retrieving resources from web servers.

- Heavyweight and stateless:

HTTP is a relatively heavy protocol, carrying a significant amount of header information with each request and response. It is designed for more powerful devices and full-fledged web applications.

- Synchronous communication:

HTTP operates synchronously, meaning that a client typically waits for the server's response before proceeding further. This synchronous nature is well-suited for web applications where immediate responses are expected.

WebSockets

- Full-duplex communication:

WebSockets enable full-duplex communication between a client and a server, allowing both parties to send and receive data simultaneously. It provides a persistent connection that remains open, facilitating real-time communication.

- Overlapping communication:

Unlike HTTP, where a new request is made for each server response, WebSockets eliminate the need for repeated request/response cycles. This makes it suitable for applications requiring continuous, bidirectional communication.

- Protocol flexibility:

WebSockets can use different protocols, such as WebSocket over HTTP or WebSocket Secure (WSS) over HTTPS, providing options for secure communication.

© Algonquin College 2022

MQTT API Design in Flask

The publisher and subscriber roles form the fundamental communication model in MQTT, known as the publish/subscribe model. It allows decoupled and asynchronous communication between different clients connected to an MQTT broker.

In the code below, (see attachment), the publisher publishes temperature values to the "temperature" topic, while the subscriber subscribes to the same topic and receives and displays the temperature values. This enables real-time temperature data exchange between the publisher and subscriber using MQTT.

Publisher:

1. The publisher code is written in Flask, a Python web framework.
2. It imports the necessary libraries, including Flask and paho.mqtt.client for MQTT functionality.
3. The broker address is set to "broker.hivemq.com", and the topic is set to "temperature".
4. The publisher creates an MQTT client and connects it to the broker.
5. There's a publish_temperature() function that runs in a loop and publishes the temperature value (25.0) to the specified topic every second.
6. The Flask app has a route "/publish" that starts the temperature publishing by setting the publishing flag to True and creating a separate thread for the publish_temperature() function.
7. The Flask app is run with the app.run(debug=True) command.

```python
from flask import Flask, jsonify
import paho.mqtt.client as mqtt
import threading
import time

app = Flask(__name__)

broker_address = "broker.hivemq.com"
topic = "temperature"
client = mqtt.Client()
client.connect(broker_address, 1883)

publishing = True


def publish_temperature():
    global publishing
    while publishing:
        temp = 25.0
        client.publish(topic, str(temp))
        print("Published temperature:", temp)
        time.sleep(1)  # Publish every second


@app.route("/publish", methods=["GET"])
def start_publishing():
    global publishing
    if not publishing:
        publishing = True
        thread = threading.Thread(target=publish_temperature)
        thread.start()
        return jsonify({"message": "Temperature publishing started."})
    else:
        return jsonify({"message": "Temperature publishing already in progress."})


if __name__ == "__main__":
    app.run(debug=True)
```

Subscriber

1. The subscriber code imports the paho.mqtt.client library for MQTT functionality.
2. The broker address is set to "broker.hivemq.com", and the topic is set to "temperature".
3. There are two functions defined: on_connect() and on_message().
4. The on_connect() function is called when the subscriber connects to the broker and prints the connection result.

5. The on_message() function is called when a message is received on the subscribed topic, and it prints the received temperature value.

6. The subscriber creates an MQTT client, sets the on_connect() and on_message() callbacks, and connects to the broker.

7. The client.loop_start() command starts the MQTT client loop to process incoming messages.

8. The while loop at the end keeps the main thread running to continue the execution of the application.

Subscriber.py > ...

```python
import paho.mqtt.client as mqtt

broker_address = "broker.hivemq.com"
topic = "temperature"

def on_connect(client, userdata, flags, rc):
    print("Connected with result code " + str(rc))
    client.subscribe(topic)

def on_message(client, userdata, msg):
    print("Received temperature:", msg.payload.decode())

client = mqtt.Client()
client.on_connect = on_connect
client.on_message = on_message

client.connect(broker_address, 1883)
client.loop_start()

# Keep the main thread running to continue the execution of your application
while True:
    pass
```

To run the code:

1. Save the publisher code in a file, e.g., publisher.py, and the subscriber code in another file, e.g., subscriber.py.

2. Make sure you have the necessary dependencies installed, including Flask and paho.mqtt.client.

3. Open a terminal or command prompt and navigate to the directory where the files are located.

4. To run the publisher, execute the command: python publisher.py.

5. The Flask app will start running, and you should see output similar to:

Running on http://127.0.0.1:5000/ (http://127.0.0.1:5000/) (Press CTRL+C to quit)

1. Open another terminal or command prompt in the same directory.

2. To trigger the temperature publishing, you can use a web client like cURL or a web browser.

3. Using a web browser: Enter **http://127.0.0.1:5000/publish (http://127.0.0.1:5000/publish)** in the address bar and press Enter.

4. The publisher will start publishing the temperature value every second, and you will see the output in the terminal where the publisher is running.

5. In the terminal where the subscriber is running, you will see the received temperature value printed whenever a message is received on the subscribed topic.

6. The publisher and subscriber should be running simultaneously for successful communication. Ensure that you have a working internet connection to connect to the MQTT broker specified in the code (broker.hivemq.com).

Note: The complete code for both the publisher and subscriber is available in the accompanying zip file.

- **Download .zip here (Module%208%20code.zip?isCourseFile=true&ou=890980)**

© Algonquin College 2022

MQTT API Design

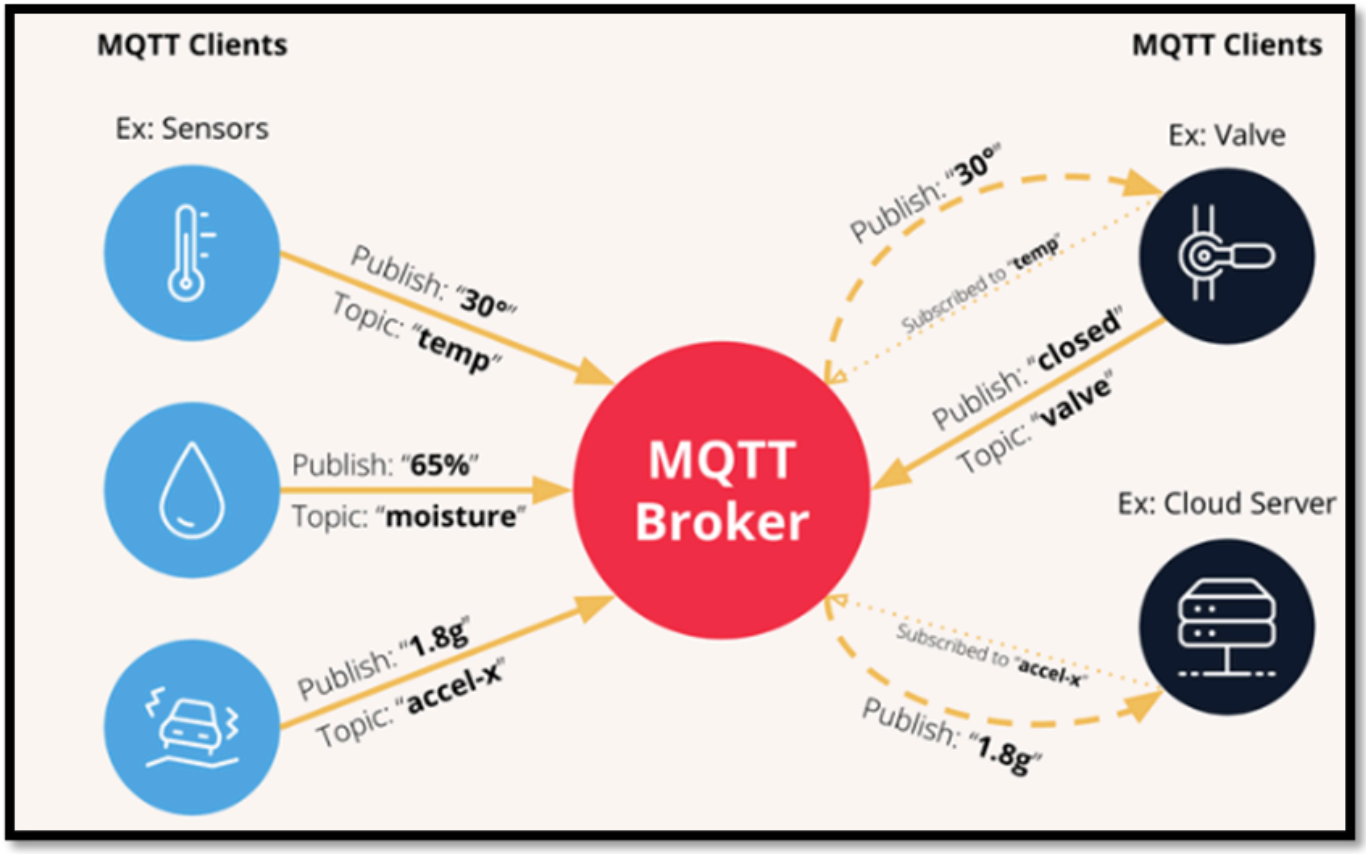
API design for MQTT involves defining the interfaces and methods that enable developers to interact with MQTT brokers and publish/subscribe to topics. The MQTT API allows clients (devices or applications) to connect to an MQTT broker, send messages, and receive messages from other clients that are subscribed to the same topics.

Following video provides a detailed introduction to MQTT.

- **https://www.youtube.com/watch?v=kuyCd53AOtg (https://www.youtube.com/watch?v=kuyCd53AOtg)**

Here are some key considerations for designing an MQTT API:

- MQTT clients: These are the devices or applications that produce or consume messages. Clients can be sensors, IoT devices, or any system that wants to send or receive data.
- Topics: Messages in MQTT are published to specific topics. Topics are like channels or categories that clients can subscribe to. They provide a way to organize and filter messages.
- Publishing: MQTT clients can publish messages to a broker by specifying a topic and the content of the message. When a client publishes a message, the broker receives it. Similarly to what is shown in the image below.



- Subscribing: MQTT clients can subscribe to specific topics on the broker. When a client

subscribes to a topic, it expresses interest in receiving messages published to that topic.

- Message delivery: When a message is published to a topic, the broker receives it and then forwards it to all the clients that have subscribed to that topic. The broker ensures that the message reaches the intended recipients.

- QoS (Quality of Service): MQTT supports different levels of QoS for message delivery, ranging from "at most once" to "at least once" and "exactly once." The QoS level determines the reliability and guarantee of message delivery.

- Broker is a central intermediary that facilitates communication between MQTT clients. It acts as a message hub, receiving messages published by clients and delivering them to clients that have subscribed to specific topics.

© Algonquin College 2022

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe messaging protocol commonly used in the Internet of Things (IoT) and other resource-constrained environments. MQTT is designed to be simple, efficient, and reliable, making it ideal for communication between devices with limited processing power and network bandwidth.

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain MQTT API design

02 Differentiate between MQTT, HTTP and WebSockets

03 Design a MQTT API

04 Implement a MQTT API

Key Terms and Concepts

- API: Application Programming Interface
- GraphQL: Graph Query Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- REST: Representational State Transfer
- SOAP: Simple Object Access Protocol
- XML: eXtensible Markup Language

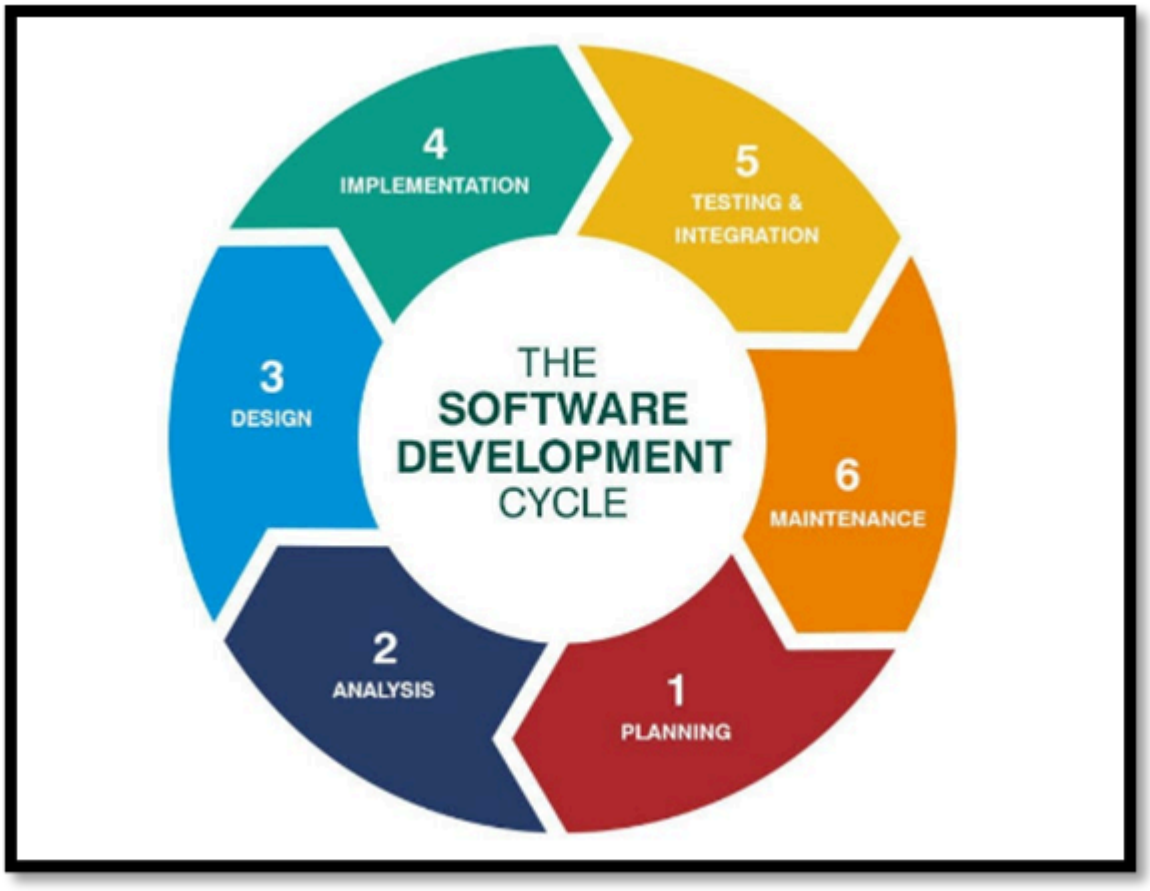
Software Development Life Cycle (SDLC)

The Software Development Life Cycle (SDLC) is a framework that provides a structured approach to software development. It consists of a series of phases, each with specific activities and deliverables.

The phases of SDLC include:

1. Planning
2. Analysis
3. Design
4. Implementation
5. Testing & Integration
6. Maintenance

Illustrated below is the software development cycle.



Let's discuss each phase of the cycle in greater detail.

1. Requirements Gathering/Planning: This phase involves gathering and analyzing requirements from stakeholders, users, and business representatives. The goal is to understand the needs

and objectives of the software project.

2. Analysis: It involves understanding user needs, identifying system functionalities, and determining project feasibility, forming the foundation for successful software development.

3. System Design: In this phase, the system architecture, components, and interfaces are designed. It includes creating high-level and low-level designs, defining data structures, and determining the overall system structure.

4. Implementation: The implementation phase involves coding or developing the software based on the design specifications. Programmers write the source code, following coding standards and best practices.

5. Testing: Testing is a critical phase to ensure the quality and functionality of the software. It includes various testing activities such as unit testing, integration testing, system testing, and user acceptance testing. Bugs and issues are identified, reported, and resolved during this phase.

6. Maintenance: After deployment, the software enters the maintenance phase, where updates, bug fixes, and enhancements are implemented. It involves monitoring the software's performance, addressing user feedback, and making necessary modifications.

Throughout the SDLC, communication and documentation play a crucial role. Clear and effective communication ensures that all stakeholders are on the same page, and any changes or updates are communicated promptly. Documentation, such as requirements documents, design specifications, test plans, and user manuals, helps in capturing and preserving project knowledge, facilitating collaboration, and maintaining accountability.

© Algonquin College 2022

Agile Methodologies

Application Development Workflow

The Application Development Workflow refers to the systematic process followed to develop, test, deploy, and maintain software applications. It involves a series of steps and activities that ensure efficient and effective development of applications.

Agile Methodologies

Agile methodologies are a set of project management and software development practices that prioritize flexibility, collaboration, and iterative development. They aim to deliver high-quality products efficiently by breaking down large projects into smaller, manageable tasks. Some popular agile methodologies include Scrum, Kanban, and Extreme Programming (XP).

For a greater overview of Agile development, please watch the following video.

- **https://www.youtube.com/watch?v=QLvBK9stdoM (https://www.youtube.com/watch?v=QLvBK9stdoM)**

Benefits

The benefits of agile methodologies include:

1. **Flexibility:**

Agile methodologies embrace change and allow for adjustments in requirements, priorities, and scope throughout the development process. This flexibility ensures that the final product meets the evolving needs of the stakeholders.

2. **Collaboration:**

Agile methodologies emphasize close collaboration between team members, stakeholders, and customers. Regular communication and feedback loops enable transparency, promote shared understanding, and foster a sense of collective ownership.

3. **Iterative Development:**

Agile approaches focus on delivering working software in short iterations or sprints. This iterative development allows for early and continuous feedback, enabling teams to quickly adapt, address issues, and make improvements.

4. **Customer Satisfaction:**

By involving customers and stakeholders in the development process, agile methodologies ensure that the delivered product meets their expectations. Regular demonstrations and feedback cycles help in aligning the development efforts with customer needs, leading to higher satisfaction.

5. **Faster Time-to-Market:**

Agile methodologies promote a steady and efficient pace of development, resulting in faster delivery of valuable features and functionalities. This accelerated time-to-market provides a competitive edge and enables businesses to respond swiftly to changing market conditions.

© Algonquin College 2022

The application development workflow refers to the process and series of steps followed to develop and deliver a software application. It outlines the sequence of activities and tasks involved in transforming an idea or concept into a fully functional and deployable software product. In this module, we'll examine the application development workflow, including agile methodologies and their benefits. Furthermore this module will require you to have an in-depth look at the Software Development Life Cycle (SDLC).

Learning Objectives

By the end of this module, you should be able to complete the following:

01 Explain Agile methodologies and their benefits

02 Recognize the importance of clear communications and accountability

03 Create application development workflow artifacts

Key Terms and Concepts

- API: Application Programming Interface
- GraphQL: Graph Query Language
- HTTP: Hypertext Transfer Protocol
- IoT: Internet of Things
- JSON: JavaScript Object Notation
- REST: Representational State Transfer
- SOAP: Simple Object Access Protocol
- XML: eXtensible Markup Language

Agile and SDLC

The Software Development Life Cycle (SDLC) and Agile methodologies can be interconnected. While SDLC follows a linear sequence of phases, Agile methodologies break down development into iterative sprints. Incorporating Agile principles within SDLC allows for flexibility and adaptability to changing requirements. Agile practices enhance collaboration and communication, fostering transparency and efficiency in SDLC.

By adopting Agile principles, SDLC embraces continuous improvement and learning, refining processes and enhancing software quality. Integrating Agile principles into SDLC ensures a customer-centric approach by involving customers throughout the development process and delivering working software early and frequently. This integration allows for a more adaptable and efficient software development process.

© Algonquin College 2022